DuckDB Studio

POWERED BY LEXIS+



GUIDE DE L'UTILISATEUR

Concevoir, interroger et optimiser vos données

L'atelier DuckDB écrit en Lexis+ : connexions à toutes vos sources, éditeur SQL, profils d'exécution, recommandations d'expert, modèles de données, pipelines planifiés, tableaux de bord et assistant IA. Pas à pas, sur une base de démonstration d'un million et demi de ventes.

12 connecteurs

9 vues de travail

16 étapes de pipeline

3 moteurs d'assistant

Prise en main · Connexions · Requêtes · Performance · Modélisation · Pipelines · BI et assistant · Extensions

DuckDB Studio · wdlexisrt

Toutes vos données, un seul atelier

© 2026 Jean-Marc QUÉRÉ

sonaliwan.fr

Bienvenue dans DuckDB Studio

DuckDB Studio est un atelier complet autour de **DuckDB**, le moteur SQL analytique qui tient dans un fichier. Il réunit dans une seule fenêtre tout ce qu'il faut pour travailler avec des données : se connecter à des bases et à des fichiers, écrire et exécuter des requêtes, comprendre pourquoi une requête est lente et comment l'accélérer, dessiner un modèle de données, enchaîner des traitements, les planifier, et présenter les résultats.

Il est écrit en **Lexis+** et s'exécute avec le runtime **wdlexisrt** : c'est un programme ordinaire, dont vous pouvez lire et adapter les sources (`exemples/duckdbstudio`), et que vous pouvez enrichir par des **extensions** sans toucher à son code.

- **Analystes et développeurs** : un éditeur SQL confortable, des résultats immédiats, des requêtes qui joignent une base PostgreSQL, un fichier Parquet et une feuille Excel dans le même `SELECT`.
- **Administrateurs** : le catalogue de chaque source, la volumétrie, le profil d'exécution, des recommandations chiffrées (index, clés, compression, requêtes à mutualiser).
- **Concepteurs** : un modèle entité-relation qui se lit dans une base ou dans un script, et qui s'écrit en SQL ; des pipelines dessinés qui deviennent du SQL lisible.
- **Exploitants** : des travaux planifiés, chaînés, relancés en cas d'échec, suivis sur un diagramme de Gantt.

Comment lire ce guide

- Les **onglets de couleur** du bord droit indiquent le chapitre ; chaque chapitre correspond à une ou deux vues de l'atelier.
- Les encadrés sont ceux du guide Lexis+ : **À retenir**, **Pour bien comprendre**, **Astuce**, **Attention**, **Syntaxe** et **À vous de jouer**.
- Les blocs de code SQL et Lexis+ sont colorés comme dans l'éditeur. Le badge **GÉNÉRÉ** signale un texte écrit par DuckDB Studio lui-même.
- Toutes les captures ont été faites sur le projet `ventes` et sa base de démonstration : refaites les manipulations en lisant, le chapitre 1 explique comment créer cette base en un clic.

Ce qu'il faut savoir avant de commencer

- Un peu de **SQL** suffit : `SELECT`, `FROM`, `WHERE`, `GROUP BY`. Le reste s'apprend en chemin, l'assistant et les modèles de la palette aident.
- Aucune connaissance de Lexis+ n'est nécessaire pour utiliser l'atelier ; elle ne sert qu'au chapitre 8, pour écrire des extensions.
- Le dialecte SQL est celui de DuckDB : `GROUP BY ALL`, `SELECT * EXCLUDE (…)`, `read_csv()`, `ASOF JOIN`… Sa documentation (duckdb.org) complète ce guide.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

Huit domaines, un même atelier

DuckDB Studio s'organise comme l'atelier Lexis+ : un **explorateur** à gauche, des **vues** en onglets au centre, les **résultats**, les **messages** et l'**assistant** en bas. Chaque domaine ci-dessous a son chapitre.



Prise en main

Lancer l'atelier, ouvrir un projet, créer la base de démonstration, se repérer.

Chapitre 1



Connexions

Bases, fichiers et serveurs : douze connecteurs, l'explorateur, la cartographie.

Chapitre 2



Requêtes SQL

Éditeur, exécution, résultats, export, requêtes fédérées, historique.

Chapitre 3



Performance

Profils d'exécution, recommandations d'expert, mutualisation, tableau de bord.

Chapitre 4



Modélisation

Le concepteur entité-relation, l'analyse d'impact, le SQL dans les deux sens.

Chapitre 5



Pipelines

Des traitements dessinés, traduits en SQL, planifiés, chaînés et suivis.

Chapitre 6



BI et assistant

Tableaux de bord sur vos requêtes ; un assistant qui explique et écrit le SQL.

Chapitre 7



Extensions

Vos règles, vos onglets, vos menus et vos connecteurs, sans toucher au code.

Chapitre 8

Je veux…	Où ?
interroger un fichier CSV ou Parquet	2.3 (connexion « fichiers ») ou 3.4 (`read_csv`)
savoir pourquoi une requête est lente	4.1 (profil, F6) puis 4.3 (recommandations)
documenter une base existante	5.1 (schéma) et 2.8 (cartographie)
alimenter une table tous les soirs	6.3 (pipeline) puis 6.8 (travail quotidien)
présenter des chiffres	7.1 (tableau de bord BI)
écrire une requête sans connaître le schéma	7.4 (assistant)

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

Sommaire

1. Prise en main 5

2. Connexions et exploration 11

3. Écrire et exécuter du SQL 18

4. Mesurer et optimiser 23

5. Modéliser les données 29

6. Pipelines et planification 32

7. Tableaux de bord et assistant 39

8. Extensions et connecteurs 44

Annexe A · Référence 49

Annexe B · Aides-mémoires 54

À retenir

Le détail des sections figure en tête de chaque chapitre. Pour une première lecture : le chapitre 1, puis les sections 2.3, 3.2, 4.1 et 4.3 — en une demi-heure, vous interrogez vos propres données et vous savez lire un profil.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

CHAPITRE 1 · PRISE EN MAIN

Premiers pas dans DuckDB Studio

Lancez l'atelier, ouvrez un projet, créez la base de démonstration et faites le tour de la fenêtre : en dix minutes, vous exécutez vos premières requêtes.



1.1 Lancer l'atelier · 1.2 Le projet · 1.3 La base de démonstration · 1.4 Visite de la fenêtre · 1.5 La barre d'outils · 1.6 Les menus · 1.7 Raccourcis · 1.8 Réglages

Dans ce chapitre, vous apprendrez à…

- lancer DuckDB Studio sur un dossier de projet
- comprendre ce que contient un projet
- créer la base de démonstration et ses connexions
- vous repérer dans la fenêtre
- retrouver une commande dans les menus
- gagner du temps avec les raccourcis

1.1 Lancer l'atelier

DuckDB Studio est un programme Lexis+ livré avec wdlexis+ dans `exemples/duckdbstudio`. Il s'exécute avec le runtime **wdlexisrt**, qui contient l'interface graphique et le moteur DuckDB :

SYNTAXE **Lancer DuckDB Studio** terminal

```
wdlexisrt exemples/duckdbstudio/principal.prg             && dernier projet ouvert
wdlexisrt exemples/duckdbstudio/principal.prg ~/Projets/ventes   && ce dossier de projet
```

Sans argument, l'atelier rouvre le **dernier projet** utilisé ; la toute première fois, il crée et ouvre le dossier `~/DuckDBStudio`. Depuis l'atelier wdlexis+, ouvrez le dossier `exemples/duckdbstudio` comme projet et exécutez-le (`Ctrl+R`) : le résultat est le même.

Un moteur, toutes les sources

Au démarrage, DuckDB Studio ouvre une base DuckDB **en mémoire** : c'est son moteur. Chaque connexion de votre projet y est ensuite **attachée** sous son nom (`demo`, `compta`…). Une même requête peut donc lire plusieurs bases et plusieurs fichiers à la fois : c'est DuckDB qui fait le travail.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

1.2 Le projet

Un **projet** est un dossier. DuckDB Studio y range tout ce qui concerne votre travail : la liste des connexions, l'historique des requêtes, les pipelines, les travaux planifiés, les tableaux de bord, les catalogues mis en cache et vos scripts SQL.

SYNTAXE Le dossier d'un projet — ~/Projets/ventes

```
ventes/                  dossier du projet
  duckstudio.json        connexions (sans secrets), historique, pipelines, travaux, BI…
  .duckstudio/           catalogues en cache (catalogue_<connexion>.json)
  *.sql                  scripts du projet (les six premiers s'ouvrent au chargement)
  connecteurs/           vos connecteurs (facultatif)
  extensions/            vos extensions .prg (facultatif)
  demo.duckdb, fichiers/ base de démonstration (Fichier › Créer la base de démonstration)
```

- *Fichier › Ouvrir un projet…* change de dossier (il est créé s'il n'existe pas).
- Le projet est enregistré automatiquement à chaque modification et à la fermeture ; *Fichier › Enregistrer le projet* le force.
- Les scripts `.sql` du dossier apparaissent sous **Scripts** dans l'explorateur ; les six premiers s'ouvrent dans l'éditeur au chargement.

> **Attention**
>
> Les **mots de passe** et **jetons** des connexions ne sont jamais écrits dans `duckstudio.json` : ils restent en mémoire pendant la session et sont redemandés à la connexion suivante. Vous pouvez donc partager un projet (ou le mettre sous Git) sans divulguer de secret.

1.3 La base de démonstration

Pour découvrir l'atelier sans risque, créez la base de démonstration : *Fichier › Créer la base de démonstration*. En quelques secondes, DuckDB Studio prépare dans le projet :

Élément	Contenu
`demo.duckdb`	50 000 **clients**, 2 000 **produits**, 1,5 million de **ventes** (clés primaires et étrangères), un **journal** sans clé, deux vues : `ventes_detail` et `ca_mensuel`
`fichiers/objectifs.csv`	les objectifs mensuels de chiffre d'affaires par catégorie de produits
`demo.sql`	cinq requêtes commentées : agrégation, tri, requête fédérée, produit cartésien, balayage complet
connexion `demo`	le fichier DuckDB, attaché sous le nom `demo`
connexion `objectifs`	le dossier `fichiers` : chaque CSV devient une vue

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

EXEMPLE **Les requêtes de démonstration** demo.sql

```sql
-- DuckDB Studio : requêtes de démonstration
-- F5 exécute la sélection (ou tout le script), F6 profile la requête sous le curseur.

-- 1. Chiffre d'affaires par ville
SELECT c.ville, count(*) AS ventes, sum(v.montant) AS ca
FROM demo.ventes v JOIN demo.clients c ON c.id = v.client_id
GROUP BY c.ville ORDER BY ca DESC;

-- 2. Meilleurs produits (tri complet : voir le profil)
SELECT p.libelle, p.categorie, sum(v.quantite) AS qte, sum(v.montant) AS ca
FROM demo.ventes v JOIN demo.produits p ON p.id = v.produit_id
GROUP BY ALL ORDER BY ca DESC LIMIT 20;

-- 3. Requête fédérée : base DuckDB + fichier CSV
SELECT m.mois, m.categorie, m.ca, o.objectif, round(100 * m.ca / o.objectif, 1) AS atteinte
FROM demo.ca_mensuel m JOIN objectifs.objectifs o
  ON o.mois = m.mois AND o.categorie = m.categorie
ORDER BY m.mois, m.categorie;

-- 4. Produit cartésien involontaire (le moteur d'expertise le signale)
SELECT count(*) FROM demo.clients c, demo.produits p WHERE c.ville = 'Lyon' AND p.prix > 400;

-- 5. Balayage complet sans filtre
SELECT avg(montant) FROM demo.ventes;
```

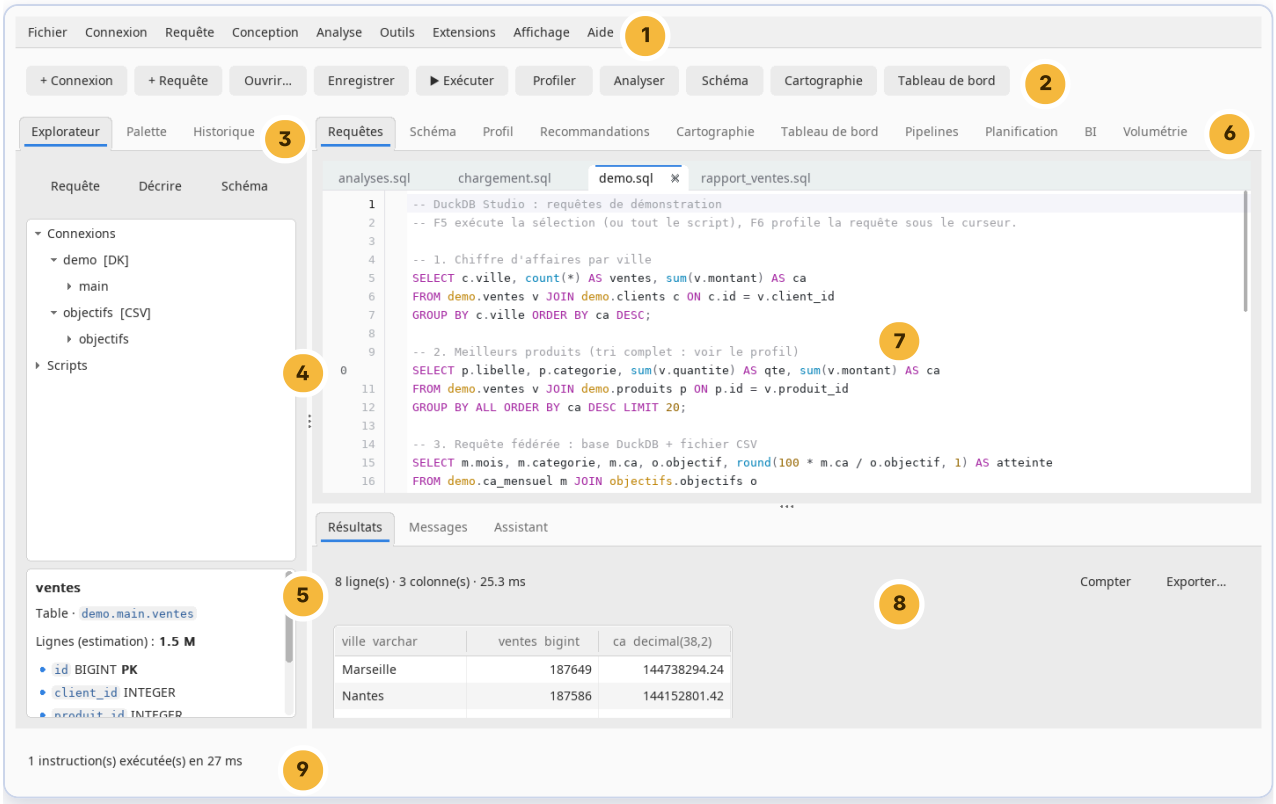
À vous de jouer

Créez la base de démonstration, placez le curseur dans la requête 1 et appuyez sur `F5`. Sélectionnez ensuite les trois lignes de la requête 3 et appuyez de nouveau sur `F5` : vous venez de joindre une base DuckDB et un fichier CSV.

1.4 Visite de la fenêtre

Voici DuckDB Studio sur le projet `ventes`, après l'exécution de la première requête de démonstration ; la table `ventes` est sélectionnée dans l'explorateur.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l



1. **Menus** : toutes les commandes (1.6)
2. **Barre d'outils** : les commandes courantes (1.5)
3. **Explorateur**, **Palette**, **Historique**
4. **Arbre** des connexions, schémas, tables et vues
5. **Fiche** de l'élément sélectionné
6. **Vues** : Requêtes, Schéma, Profil… (une par chapitre)
7. **Éditeur SQL**, un onglet par script
8. **Résultats**, **Messages**, **Assistant**
9. **Barre d'état** : durée, nombre de lignes, erreurs

Les séparateurs entre les zones se déplacent à la souris. Les vues du centre sont :

Vue	Rôle	Chapitre
Requêtes	l'éditeur SQL et ses scripts	3
Schéma	le modèle entité-relation d'un schéma ou d'un script	5
Profil	le plan d'exécution mesuré de la dernière requête profilée	4
Recommandations	les conseils du moteur d'expertise, avec leur correction	4
Cartographie	toutes les sources, leurs tables et les vues qui en dépendent	2
Tableau de bord	inventaire, statistiques d'exécution, score global	4
Pipelines	le concepteur de traitements	6
Planification	les travaux, leurs déclencheurs et leur journal	6
BI	les tableaux de bord de restitution	7

Les extensions peuvent ajouter leurs propres vues (chapitre 8) : sur les captures, l'onglet **Volumétrie** vient de l'extension d'exemple.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

1.5 La barre d'outils

Bouton	Effet
+ Connexion	ouvre le concepteur de connexions (2.3)
+ Requête	nouveau script SQL dans l'éditeur
Ouvrir… · Enregistrer	ouvre un script `.sql` · enregistre le script actif dans le projet
▶ Exécuter	exécute la sélection, ou tout le script (**F5**)
Profiler	mesure la requête sous le curseur (**F6**) et passe à la vue Profil
Analyser	lance le moteur d'expertise sur le plan, le catalogue et l'historique
Schéma · Cartographie · Tableau de bord	affichent la vue correspondante

1.6 Les menus

Menu	Commandes
Fichier	nouvelle requête, ouvrir et enregistrer un script, ouvrir et enregistrer le projet, créer la base de démonstration, quitter
Connexion	nouvelle connexion, modifier, tester, supprimer la connexion sélectionnée, relire tous les catalogues
Requête	exécuter, profiler, compter les lignes du résultat, exporter le résultat, commenter, effacer l'historique
Conception	schéma de la connexion sélectionnée, script → modèle, modèle → SQL, nouvelle table, analyse d'impact
Analyse	analyser le projet, générer la correction, rétablir les recommandations ignorées
Outils	pipelines, travaux planifiés et planificateur, éléments BI, assistant et ses réglages
Extensions	gestionnaire, nouvelle extension, recharger ; puis les commandes des extensions
Affichage	chacune des vues, thème sombre ou clair
Aide	raccourcis clavier, connecteurs disponibles, à propos

1.7 Raccourcis clavier

Touches	Effet
F5 ou **Ctrl+R**	exécuter la sélection, ou tout le script
F6	profiler la requête sous le curseur (EXPLAIN ANALYZE)
Ctrl+Espace	complétion : tables, colonnes après `table.`, mots clés, fonctions

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

Touches	Effet
Ctrl+B	commenter ou décommenter les lignes sélectionnées (`--`)
Ctrl+S · Ctrl+N · Ctrl+O	enregistrer le script · nouvelle requête · ouvrir un script
Ctrl+Tab	script suivant
Ctrl++ · Ctrl+- · Ctrl+0	agrandir, réduire la police de l'éditeur, taille par défaut
Suppr · Échap	dans un diagramme : supprimer · désélectionner
molette	dans un diagramme : zoom ; glisser le fond : déplacer la vue

1.8 Réglages

Les réglages personnels sont enregistrés dans `~/.duckdbstudio.json` : le dernier projet, le thème (*Affichage › Thème sombre / clair*), la taille de la police de l'éditeur, la **limite** de lignes affichées (1 000 par défaut) et le moteur de l'assistant (7.6). Ils ne dépendent pas du projet.

> **Astuce**
>
> La limite ne concerne que l'**affichage** : **Compter** donne le nombre exact de lignes et **Exporter…** écrit le résultat complet. Pour afficher davantage de lignes, modifiez `"LIMITE"` dans `~/.duckdbstudio.json`.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

CHAPITRE 2 · CONNEXIONS

Toutes vos sources dans un seul moteur

Bases DuckDB, SQLite, PostgreSQL, MySQL, SQL Server, serveurs Quack, ODBC, fichiers CSV, Parquet, JSON et Excel : déclarez une connexion, DuckDB Studio lit son catalogue et vous la montre.



2.1 Le principe · 2.2 Les connecteurs · 2.3 Créer une connexion · 2.4 Secrets · 2.5 L'explorateur · 2.6 La palette · 2.7 Fichiers · 2.8 La cartographie · 2.9 Cas particuliers

Dans ce chapitre, vous apprendrez à…

- ✔ choisir le connecteur adapté à une source
- ✔ protéger les mots de passe
- ✔ interroger un dossier de fichiers comme une base
- ✔ créer, tester et modifier une connexion
- ✔ parcourir tables, vues et macros dans l'explorateur
- ✔ lire la cartographie des dépendances

2.1 Le principe

Une **connexion** a un nom, un type (son **connecteur**) et des paramètres. Quand elle est attachée, DuckDB Studio exécute pour elle une instruction `ATTACH` (ou crée des vues pour des fichiers), puis lit son catalogue : schémas, tables, colonnes, clés, index, vues et macros.

GÉNÉRÉ **Ce que fait DuckDB Studio pour vous** — SQL exécuté à l'attachement

```
ATTACH 'demo.duckdb' AS demo;                                  -- DuckDB
ATTACH 'host=srv port=5432 dbname=compta user=lecteur password=…'
       AS compta (TYPE postgres, READ_ONLY);                   -- PostgreSQL
ATTACH 'quack:entrepot:9494' AS entrepot (TOKEN '…');          -- Quack
CREATE VIEW memory.objectifs.objectifs AS
  SELECT * FROM read_csv_auto('…/fichiers/objectifs.csv');     -- CSV
```

Dans vos requêtes, un objet se désigne par **connexion.schéma.objet** : `demo.main.ventes`, ou simplement `demo.ventes` quand le schéma est `main`. Le catalogue est gardé en cache dans `.duckstudio/` : l'explorateur s'affiche même avant la reconnexion d'une source distante.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

2.2 Les connecteurs

Connecteur	Paramètres	Remarques
DuckDB (fichier)	fichier `.duckdb` ; option lecture seule	natif
DuckDB (mémoire)	aucun	base de travail vide
Quack	hôte, port (9494), jeton ; option sans TLS	serveur DuckDB distant ; DuckDB 2.0 ou plus
SQLite	fichier	extension `sqlite`
PostgreSQL	hôte, port, base, utilisateur, mot de passe ; lecture seule	extension `postgres`
MySQL / MariaDB	hôte, port, base, utilisateur, mot de passe ; lecture seule	extension `mysql`
SQL Server	serveur, base, utilisateur, mot de passe ; certificat de confiance	extension communautaire `mssql`
ODBC	chaîne de connexion ou DSN	extension communautaire `nanodbc` ; mode requête
Fichiers CSV	dossier, motif (`*.csv`)	une vue par fichier
Fichiers Parquet	dossier, motif (`*.parquet`)	une vue par fichier
Fichiers JSON	dossier, motif (`*.json`)	une vue par fichier
Classeurs Excel	dossier, motif (`*.xlsx`)	extension `excel`

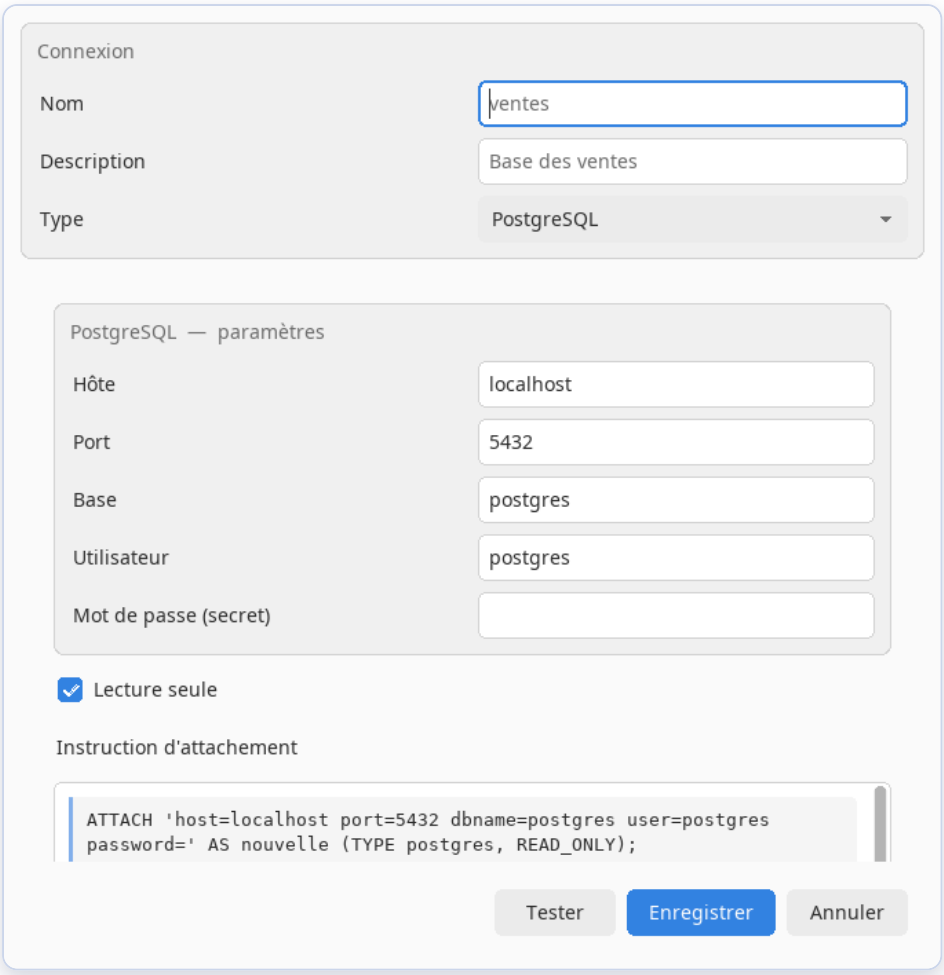
Les extensions DuckDB

Les connecteurs autres que DuckDB et les fichiers CSV, Parquet et JSON reposent sur une **extension** de DuckDB. DuckDB Studio la charge à l'attachement ; la première fois, il la télécharge depuis le dépôt de DuckDB (une connexion Internet est alors nécessaire). La liste est aussi dans *Aide › Connecteurs disponibles*.

2.3 Créer une connexion

Cliquez sur **+ Connexion** (ou double-cliquez une source dans la palette). Le concepteur de connexions s'adapte au type choisi : il affiche ses paramètres, ses options et, en direct, l'instruction d'attachement qui sera exécutée.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l



1. Donnez un **nom** court : il devient le nom SQL de la connexion (minuscules, sans accent ni espace : « Ventes 2024 » devient `ventes_2024`).
2. Choisissez le **type**, puis remplissez les paramètres ; le bouton **…** choisit un fichier ou un dossier.
3. Cliquez sur **Tester** : DuckDB Studio attache la source, mesure un aller-retour (*ping*) et compte ses tables et ses vues, sans rien enregistrer.
4. Cliquez sur **Enregistrer** : la connexion rejoint le projet, son catalogue s'affiche dans l'explorateur.

Pour modifier une connexion, sélectionnez-la dans l'explorateur puis *Connexion › Modifier la connexion…* ; *Tester la connexion* mesure de nouveau le temps de réponse, *Supprimer la connexion* la détache et l'oublie.

2.4 Secrets et sécurité

Les paramètres marqués **(secret)** — mots de passe, jetons — sont demandés à l'attachement et conservés **en mémoire** seulement. À l'ouverture d'un projet, les connexions sans secret sont attachées tout de suite ; les autres le sont quand vous relisez les catalogues (↻) ou ouvrez la connexion, après la saisie du mot de passe. En attendant, leur catalogue en cache reste consultable.

> **Astuce**
> Pour une base de production, cochez **Lecture seule** : DuckDB refuse alors toute écriture sur cette connexion, même par erreur, et le moteur d'expertise ne vous proposera jamais de la modifier.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

2.5 L'explorateur

L'arbre montre chaque connexion avec son sigle (**[DK]** pour DuckDB, **[PG]** pour PostgreSQL…), ses schémas, puis **Tables**, **Vues** et **Macros** avec leur nombre ; chaque table indique son nombre de lignes estimé. La branche **Scripts** liste les fichiers `.sql` du projet.

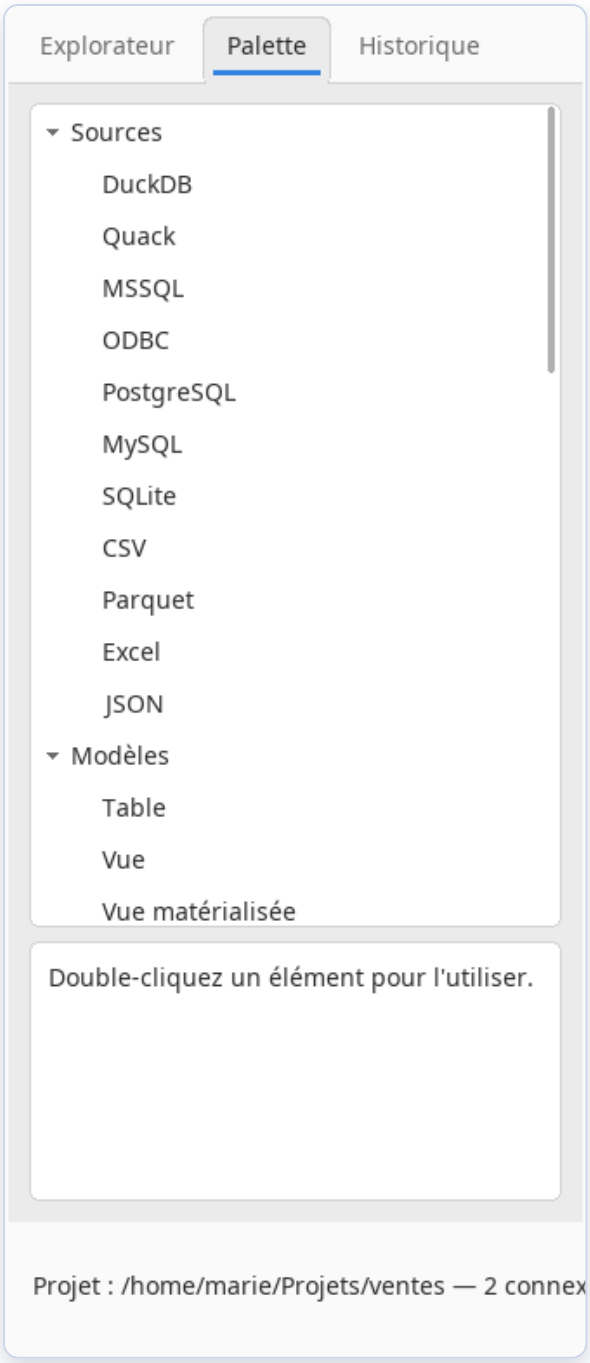
Un clic affiche la **fiche** de l'élément sous l'arbre : nom qualifié, volumétrie, colonnes avec leur type et la marque **PK**, clés étrangères, définition d'une vue. Un double-clic sur une table l'interroge, sur un schéma ouvre son modèle, sur une connexion ouvre ses paramètres, sur un script l'ouvre dans l'éditeur.

Bouton	Effet sur l'élément sélectionné
Requête	insère et exécute `SELECT * FROM … LIMIT 100`
Décrire	exécute `SUMMARIZE` : type, minimum, maximum, valeurs distinctes, moyenne, écart type, quartiles et taux de NULL de chaque colonne
Schéma	ouvre le concepteur de schéma sur ce schéma (chapitre 5)
↻	relit le catalogue de toutes les connexions

2.6 La palette d'outils

L'onglet **Palette** regroupe ce que l'on ajoute au projet : des **sources** (double-clic : nouvelle connexion de ce type), des **modèles** SQL (table, vue, vue matérialisée, macro, fonction, index : insérés dans le script actif), des **flux** (import, export, jointure, filtre, agrégation, tri, union, fenêtres analytiques, fusion), l'**orchestration** (pipeline, travail planifié), l'**optimisation** (profil, analyses) et l'**analyse** (assistant, élément BI, extensions). Un clic affiche l'aide de l'élément et son modèle SQL.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l



2.7 Les fichiers comme des tables

Une connexion de type **fichiers** (CSV, Parquet, JSON, Excel) désigne un **dossier** et un **motif**. Chaque fichier qui correspond devient une vue : `objectifs.csv` du dossier `fichiers` se lit `objectifs.objectifs`. Ajoutez un fichier dans le dossier puis ↻ : il apparaît.

Pour une lecture ponctuelle, nul besoin de connexion : les fonctions de DuckDB lisent directement un fichier, ou un ensemble de fichiers avec un motif.

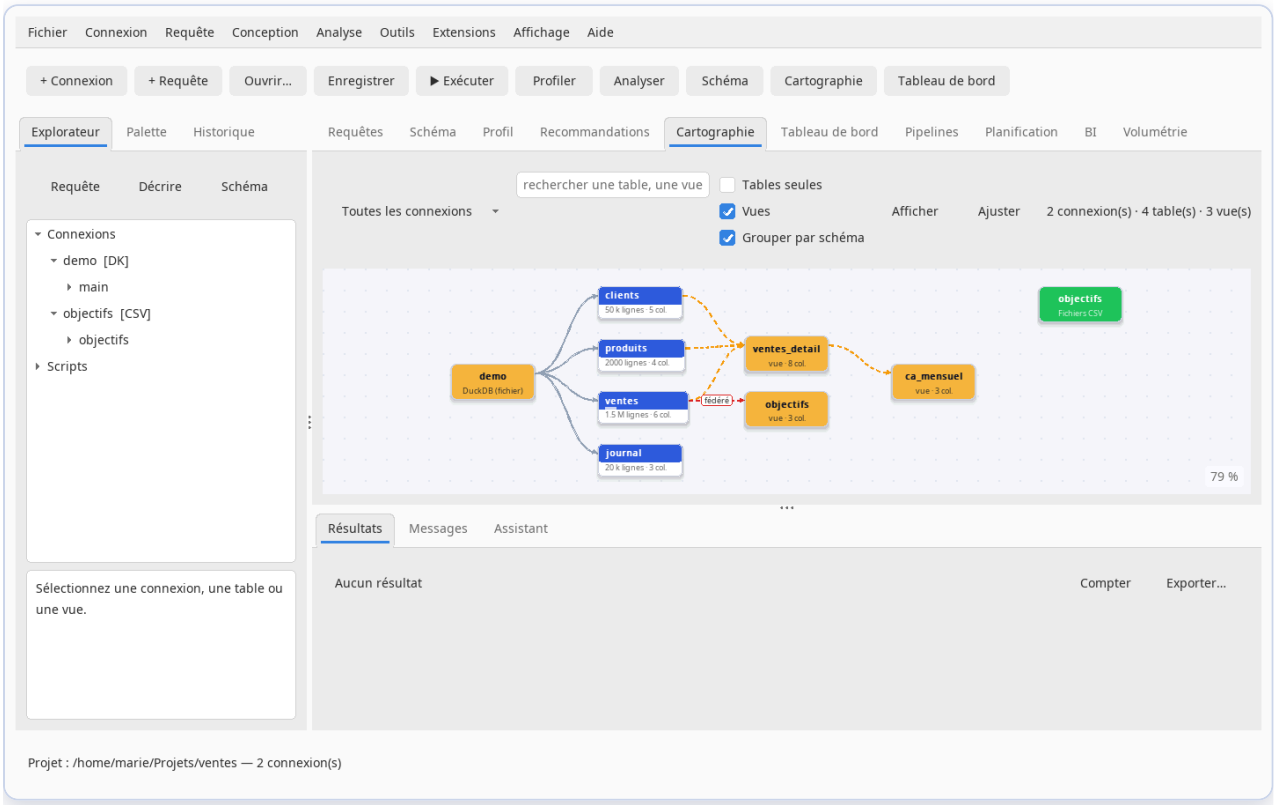
© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

EXEMPLE **Lire des fichiers sans connexion** `requete.sql`

```sql
SELECT * FROM read_csv('exports/*.csv', union_by_name = true);
SELECT count(*) FROM 'archives/2024/*.parquet';
SELECT * FROM read_json_auto('api/commandes.json');
```

2.8 La cartographie

La vue **Cartographie** dessine l'ensemble des sources : chaque connexion, ses schémas (option *Grouper par schéma*), ses tables avec leur volume, et les vues reliées aux objets qu'elles lisent. Un lien **rouge** signale une dépendance **fédérée** : une vue qui lit une autre connexion.



- Le filtre limite la carte à une connexion ; la recherche garde les tables et les vues dont le nom contient le texte saisi, avec leurs dépendances.
- Un clic affiche le nom qualifié et la définition dans la barre d'état ; un double-clic sur une table l'interroge, sur une connexion ouvre son schéma.

2.9 Cas particuliers

Source	À savoir
Quack	protocole client-serveur de DuckDB 2.0 : un serveur DuckDB partage ses bases ; le jeton fait office de mot de passe. Pour un serveur de test sans certificat, cochez *Sans TLS*.
SQL Server	l'extension communautaire est installée par `INSTALL mssql FROM community`, automatiquement à la première connexion. Cochez *Certificat de confiance* pour un serveur au certificat auto-signé.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

Source	À savoir
ODBC	mode **requête** : la source n'est pas attachée, on l'interroge par les fonctions `odbc_query(connection = …, query = …)` et `odbc_scan(table_name = …, connection = …)`.
Bases distantes	DuckDB pousse les filtres et les projections vers la source quand il le peut ; pour des traitements lourds, copiez d'abord les données (`CREATE TABLE … AS SELECT …`) ou utilisez un pipeline (chapitre 6).

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

CHAPITRE 3 · REQUÊTES SQL

Écrire et exécuter du SQL

Un éditeur qui connaît vos tables et vos colonnes, signale les erreurs pendant la frappe et exécute une sélection, une requête ou un script entier ; des résultats typés, comptés et exportés.



3.1 L'éditeur · 3.2 Exécuter · 3.3 Les résultats · 3.4 Requêtes fédérées · 3.5 Les erreurs · 3.6 L'historique · 3.7 Exporter

Dans ce chapitre, vous apprendrez à…

- ✔ profiter de la complétion des tables et des colonnes
- ✔ exécuter une sélection ou un script entier
- ✔ lire, compter et exporter un résultat
- ✔ joindre plusieurs sources dans une requête
- ✔ comprendre et corriger une erreur
- ✔ retrouver une requête dans l'historique

3.1 L'éditeur SQL

La vue **Requêtes** contient un éditeur à onglets : un onglet par script. **+ Requête** (`Ctrl+N`) en ouvre un nouveau ; **Enregistrer** (`Ctrl+S`) le range dans le dossier du projet, sous le nom de votre choix.

- **Coloration SQL** : mots clés, types, fonctions, chaînes, nombres et commentaires (`--` et `/* … */`).
- **Complétion** : après trois lettres, ou avec `Ctrl+Espace`, l'éditeur propose les tables et les vues de toutes les connexions, les mots clés, les types et les fonctions de DuckDB. Après un nom de table suivi d'un point (`ventes.`), il propose ses colonnes.
- **Commentaires** : `Ctrl+B` commente ou décommente les lignes sélectionnées.
- **Validation** : pendant la frappe, chaque requête est analysée par DuckDB lui-même ; une erreur de syntaxe est soulignée à l'endroit exact où DuckDB l'a détectée (3.5).

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

3.2 Exécuter

▶ **Exécuter**, `F5` ou `Ctrl+R` exécutent :

- la **sélection**, s'il y en a une — une requête au milieu d'un script, ou plusieurs ;
- sinon le **script entier**, instruction par instruction (séparées par `;`, chaînes et commentaires respectés).

Le résultat de la **dernière requête qui renvoie des lignes** s'affiche dans **Résultats** ; chaque instruction est journalisée dans **Messages** avec sa durée et son nombre de lignes. Après un `CREATE`, un `DROP` ou un `ALTER`, les catalogues sont relus : l'explorateur et la complétion connaissent aussitôt les nouveaux objets.

> **À retenir**
>
> Sélectionnez avant d'exécuter : un script de démonstration contient plusieurs requêtes, et seule la dernière s'afficherait. Pour mesurer une requête plutôt que l'exécuter, placez simplement le curseur dedans et appuyez sur `F6` (chapitre 4).

3.3 Les résultats

La grille affiche les colonnes dans l'ordre de la requête, avec leur **type** dans l'en-tête ; les nombres sont alignés à droite et les valeurs NULL notées **ø**. La ligne d'état donne le nombre de lignes, de colonnes et la durée.

- Au-delà de 1 000 lignes, l'affichage est limité (« Plus de 1000 lignes ») : **Compter** exécute un `count(*)` sur la requête et donne le total exact.
- Les colonnes composées (listes, structures, dictionnaires) sont affichées sous leur forme texte.

3.4 Requêtes fédérées

Toutes les connexions étant attachées au même moteur, une requête peut joindre des sources différentes. La troisième requête de démonstration compare le chiffre d'affaires de la base DuckDB aux objectifs du fichier CSV :

EXEMPLE **Une base et un fichier dans la même requête** — demo.sql

```sql
SELECT m.mois, m.categorie, m.ca, o.objectif,
       round(100 * m.ca / o.objectif, 1) AS atteinte
FROM demo.ca_mensuel m                -- connexion « demo » (fichier DuckDB)
  JOIN objectifs.objectifs o          -- connexion « objectifs » (fichiers CSV)
    ON o.mois = m.mois AND o.categorie = m.categorie
ORDER BY m.mois, m.categorie;
```

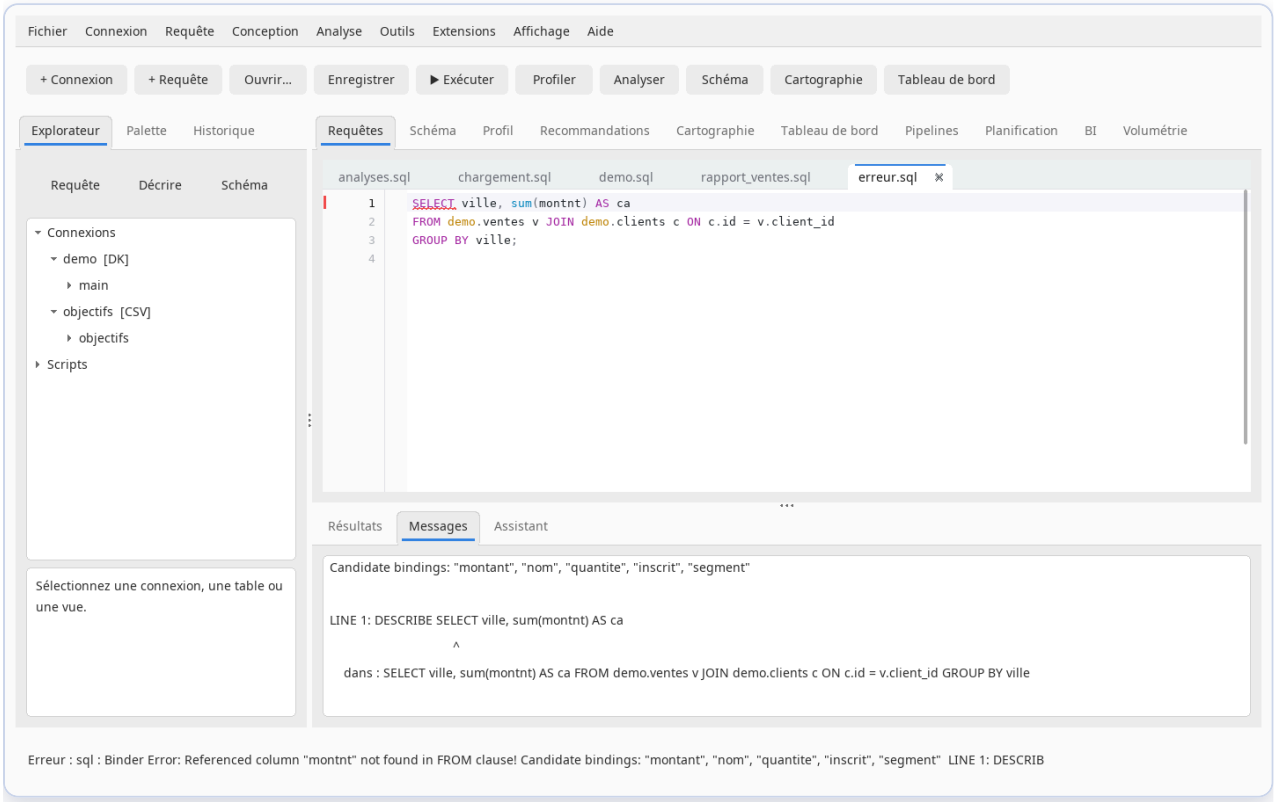
Le même principe vaut pour une table PostgreSQL et un historique en Parquet, une base SQL Server et un référentiel Excel… DuckDB lit chaque source dans son format et fait la jointure en mémoire.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

> **Astuce**
>
> Une requête fédérée relit les sources distantes à chaque exécution. Si elle sert souvent, matérialisez-la (`CREATE TABLE … AS`) et rafraîchissez-la par un travail planifié (chapitre 6) : le moteur d'expertise vous le proposera de lui-même (chapitre 4).

3.5 Les erreurs

Une erreur d'exécution s'affiche dans **Messages** (l'onglet s'ouvre de lui-même) et dans la barre d'état ; l'instruction fautive est soulignée dans l'éditeur. Ici, une faute de frappe dans un nom de colonne : DuckDB indique la ligne, la position et les colonnes candidates.



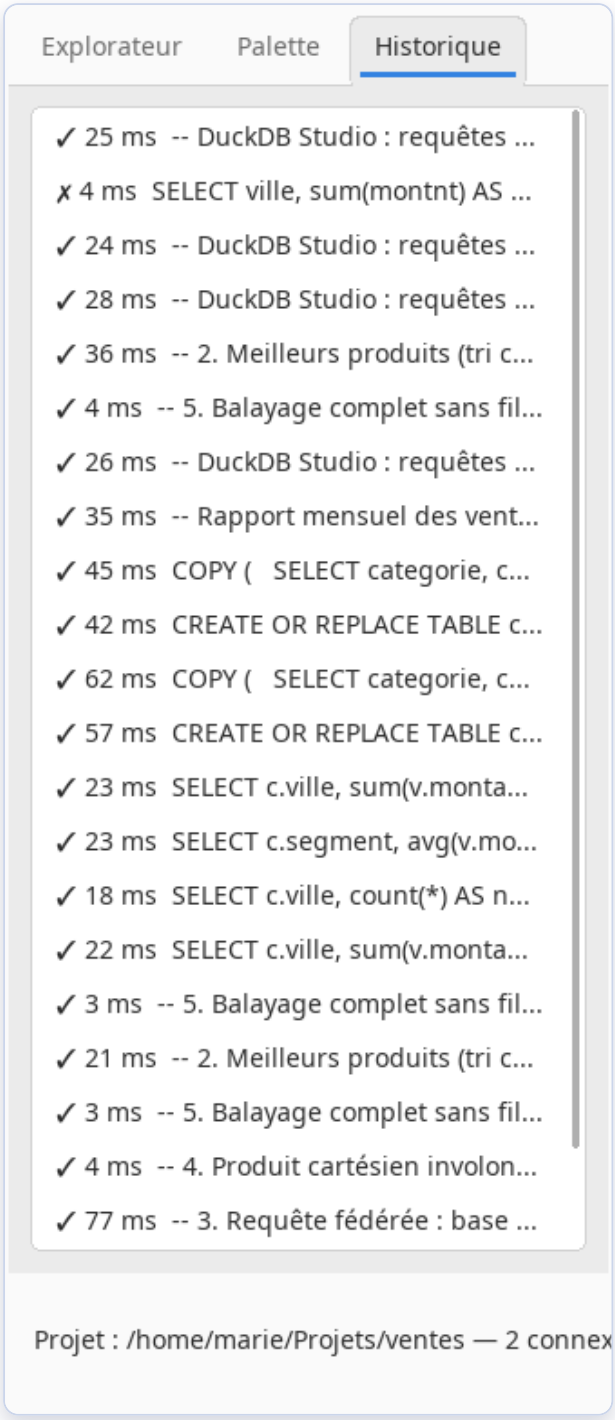
Message de DuckDB	Cause habituelle
Parser Error	syntaxe : virgule, parenthèse, mot clé ; la position est indiquée
Catalog Error … does not exist	table inconnue : oubli du préfixe de connexion (`demo.ventes`)
Binder Error … not found	colonne inconnue ou ambiguë : préfixez-la par sa table
must appear in the GROUP BY	colonne non agrégée : utilisez `GROUP BY ALL`
Conversion Error	valeur impossible à convertir : `TRY_CAST(x AS type)`

L'assistant explique la dernière erreur et propose une correction (7.5) ; sans modèle de langage, il cherche dans le catalogue les noms les plus proches de celui qui a été refusé.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

3.6 L'historique

Chaque exécution est mémorisée dans le projet (les 500 dernières) : texte, date, durée, nombre de lignes, réussite ou échec. L'onglet **Historique**, à gauche, les liste de la plus récente à la plus ancienne ; un double-clic réinsère la requête dans l'éditeur.



L'historique n'est pas qu'un aide-mémoire : il alimente le **tableau de bord** (durées, requêtes les plus coûteuses) et la **mutualisation** (requêtes répétées, jointures communes), au chapitre 4. *Requête › Effacer l'historique* le vide.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

3.7 Exporter

Exporter… écrit le résultat **complet** de la dernière requête (sans la limite d'affichage) dans un fichier **Parquet**, **CSV** (avec en-tête) ou **JSON**, selon l'extension choisie. C'est une instruction `COPY` de DuckDB : des millions de lignes s'exportent en quelques secondes.

EXEMPLE **L'équivalent en SQL** requete.sql

```sql
COPY (SELECT * FROM demo.ventes_detail WHERE jour >= DATE '2025-01-01')
  TO 'ventes_2025.parquet' (FORMAT parquet);
```

À vous de jouer

Dans un nouveau script, écrivez une requête qui donne, pour chaque segment de clients, le nombre de ventes et le panier moyen (`demo.clients` et `demo.ventes`). Exécutez-la, comptez ses lignes, puis exportez-la en CSV et ouvrez le fichier dans un tableur.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

CHAPITRE 4 · PERFORMANCE

Mesurer et optimiser

Le profil montre où le temps passe, opérateur par opérateur. Le moteur d'expertise en tire des recommandations chiffrées, et repère les traitements à mutualiser dans l'historique du projet.

4.1 Profiler · 4.2 Lire un plan · 4.3 Les recommandations · 4.4 Les règles · 4.5 La mutualisation · 4.6 Corriger · 4.7 Le tableau de bord

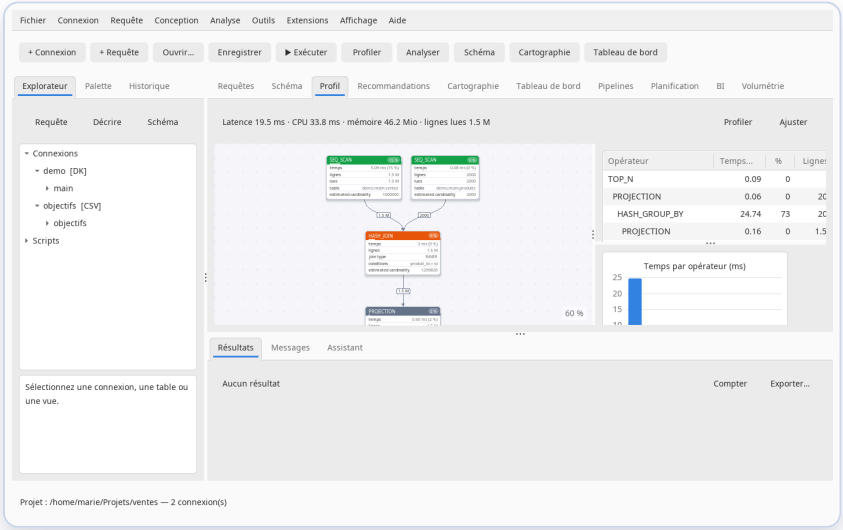


Dans ce chapitre, vous apprendrez à…

- profiler une requête et lire son plan
- comprendre les niveaux de recommandation
- mutualiser des requêtes qui refont le même travail
- repérer l'opérateur qui coûte le plus
- générer et appliquer une correction
- suivre la santé du projet

4.1 Profiler une requête

Placez le curseur dans une requête (ou sélectionnez-la) et appuyez sur `F6` ou sur **Profiler**. DuckDB Studio l'exécute sous `EXPLAIN (ANALYZE, FORMAT JSON)` : DuckDB fait le vrai travail et rend, pour chaque opérateur, le temps passé, les lignes produites et les lignes lues.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

- En haut : la **latence**, le temps **CPU** (supérieur à la latence quand DuckDB travaille sur plusieurs cœurs), la **mémoire** maximale, les **lignes lues** et, s'il y a lieu, le volume écrit sur disque.
- Au centre : le **plan**. Les données montent des feuilles (lectures) vers la racine ; chaque lien porte le nombre de lignes transmises ; la pastille donne la part du temps total, la barre en bas du nœud la matérialise.
- À droite : le même plan en **tableau**, indenté, et l'**histogramme** des huit opérateurs les plus coûteux.

GÉNÉRÉ **Ce que DuckDB Studio exécute** F6

```
EXPLAIN (ANALYZE, FORMAT JSON)
SELECT p.libelle, p.categorie, sum(v.quantite) AS qte, sum(v.montant) AS ca
FROM demo.ventes v JOIN demo.produits p ON p.id = v.produit_id
GROUP BY ALL ORDER BY ca DESC LIMIT 20;
```

4.2 Lire un plan

Les couleurs des nœuds distinguent les familles d'opérateurs :

Opérateur	Ce qu'il fait	À surveiller
TABLE_SCAN / SEQ_SCAN	lit une table, en appliquant les filtres poussés	des millions de lignes lues sans filtre
FILTER	élimine des lignes	un filtre qui aurait pu être poussé à la lecture
HASH_JOIN	jointure par table de hachage (côté construction à droite)	plus de lignes en sortie qu'en entrée
NESTED_LOOP / CROSS_PRODUCT	compare chaque ligne à chaque ligne	toujours : coût quadratique
HASH_GROUP_BY	regroupe et agrège	des groupes très nombreux
ORDER_BY / TOP_N	trie tout / garde les n premiers	un tri complet de millions de lignes
WINDOW	fonctions de fenêtre	des partitions énormes
PROJECTION	calcule les colonnes du résultat	rarement coûteux

Un clic sur un nœud affiche ses détails dans la barre d'état : table lue, conditions de jointure, filtres, agrégats, cardinalité estimée par l'optimiseur. Une forte différence entre la cardinalité **estimée** et les lignes **réelles** signale des statistiques trompeuses.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

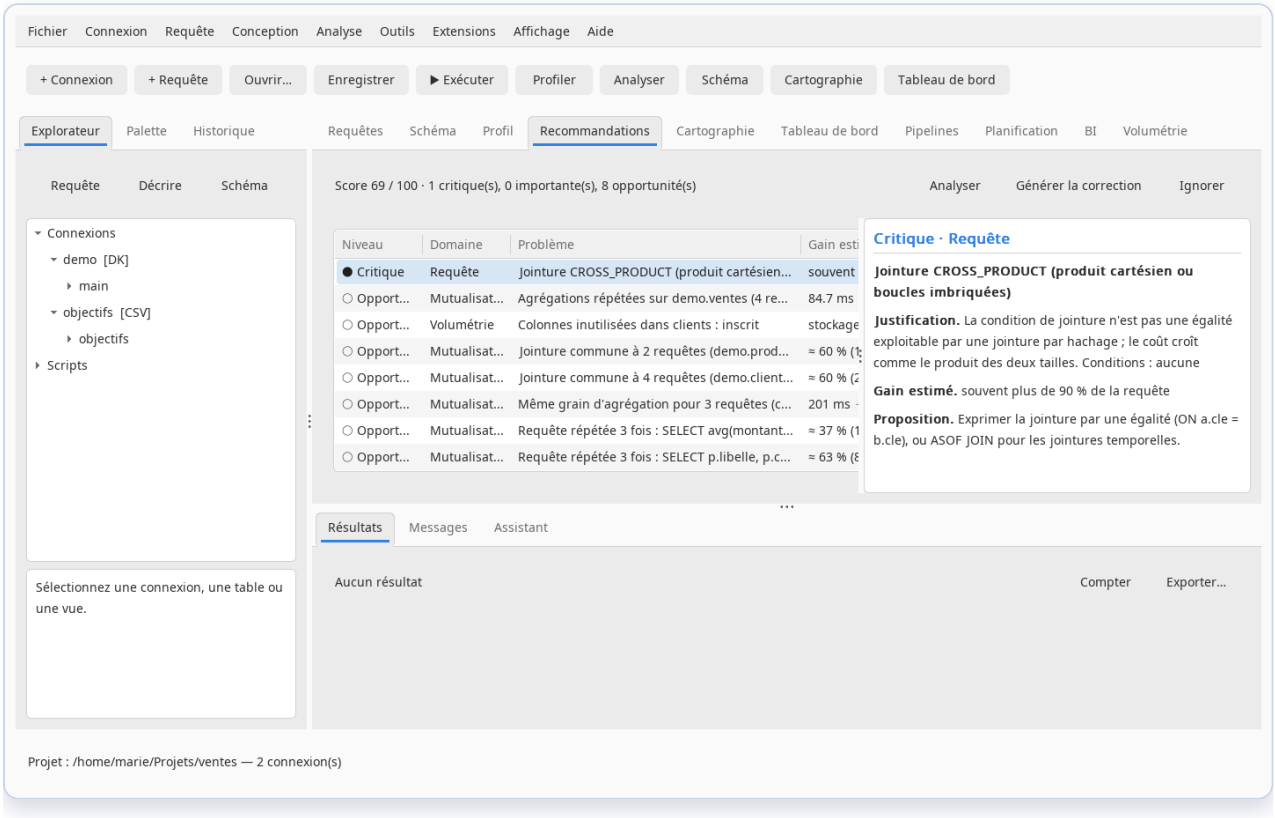
4.3 Les recommandations

Après chaque profil, et à chaque clic sur **Analyser**, le **moteur d'expertise** examine le dernier plan, les catalogues de toutes les connexions et l'historique des requêtes. Chaque recommandation a un niveau :

Critique
Un défaut qui coûte très cher ou empire avec le volume : produit cartésien, balayage de plus de dix millions de lignes.

Important
Un gain net et sûr : jointure explosive, tris répétés, débordement sur disque, fragmentation, requête lourde répétée.

Opportunité
Une amélioration à envisager : clé primaire, colonnes inutiles, compression, agrégats précalculés.



Le tableau trie les recommandations par niveau ; la fiche de droite détaille la sélection : le **problème**, sa **justification** (chiffres du plan ou du catalogue), le **gain estimé**, la **proposition** et, quand elle existe, la **correction SQL**. Pour une mutualisation, elle compare la durée actuelle et la durée attendue (retour sur investissement).

EXEMPLE **Le produit cartésien de la démonstration** — demo.sql

```sql
SELECT count(*)
FROM demo.clients c, demo.produits p        -- aucune condition de jointure
WHERE c.ville = 'Lyon' AND p.prix > 400;
```


© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

4.4 Les règles

Règle	Déclenchement	Niveau
Scan complet	un opérateur de lecture lit au moins 1 million de lignes sans filtre poussé	Important ; Critique dès 10 M
Produit cartésien	NESTED_LOOP, CROSS_PRODUCT ou BLOCKWISE_NL dans le plan	Critique
Jointure explosive	la sortie dépasse dix fois la plus grande entrée (et 100 000 lignes)	Important
Construction volumineuse	côté construction cinq fois plus grand que l'autre côté (et 1 M)	Opportunité
Tri complet	un tri de plus d'un million de lignes	Opportunité
Opérateur dominant	un opérateur prend 60 % du temps (requête de plus de 50 ms)	Opportunité
Tris répétés	deux tris ou plus dans le même plan	Important
Débordement sur disque	la requête a écrit des données temporaires	Important
Table très volumineuse	au moins 10 millions de lignes	Important
Clé primaire absente	table d'au moins 100 000 lignes sans clé primaire	Opportunité
Structures redondantes	deux tables aux colonnes identiques	Opportunité
Colonnes inutilisées	colonnes jamais citées par l'historique ni par les vues	Opportunité
Fragmentation, compression	tables DuckDB d'au moins 500 000 lignes : groupes de lignes, segments non compressés	Important, Opportunité
Requête répétée	la même requête (aux valeurs près) au moins trois fois	Important dès 5 s cumulées
Agrégations, sous-requêtes répétées	trois requêtes agrègent la même table ; la même sous-requête revient	Opportunité
Jointure commune, même grain	empreintes : 4.5	Important ou Opportunité

Les extensions ajoutent leurs propres règles (chapitre 8) : celle de l'exemple signale les tables de travail oubliées et les colonnes `…_id` sans clé étrangère.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

4.5 La mutualisation

Des requêtes différentes refont souvent le même travail : la même jointure, la même agrégation au même niveau de détail. Le moteur d'expertise calcule l'**empreinte** de chaque requête de l'historique à partir de son **arbre syntaxique**, lu par DuckDB : les tables lues, les conditions de jointure et les clés de regroupement. Les requêtes qui partagent une empreinte donnent une recommandation :

- **Jointure commune** : plusieurs requêtes joignent les mêmes tables par les mêmes conditions ; DuckDB Studio propose la table de jointure à calculer une fois.
- **Même grain d'agrégation** : plusieurs requêtes regroupent les mêmes tables par les mêmes clés ; il propose une table d'agrégats à ce grain, avec la liste des requêtes concernées.

GÉNÉRÉ **Correction proposée pour une jointure commune** Recommandations

```sql
CREATE OR REPLACE TABLE jointure_1 AS
SELECT *
FROM demo.ventes AS v JOIN demo.clients AS c ON (c.id = v.client_id);
-- requêtes concernées :
--   SELECT c.ville, sum(v.montant) AS ca FROM demo.ventes v JOIN demo.clients c …
--   SELECT c.segment, avg(v.montant) FROM demo.ventes v JOIN demo.clients c …
```

Pourquoi l'arbre syntaxique ?

Deux requêtes peuvent s'écrire très différemment (alias, ordre des tables, mise en page) et faire la même jointure. En comparant l'arbre analysé par DuckDB plutôt que le texte, DuckDB Studio les reconnaît ; il retrouve aussi les requêtes répétées « aux valeurs près » (`WHERE ville = 'Lyon'` et `WHERE ville = 'Brest'`).

Le gain est une **estimation** prudente tirée des durées de l'historique : il chiffre l'intérêt de la mutualisation, sans dispenser de mesurer après coup.

4.6 Corriger, ignorer

- **Générer la correction** ouvre la correction SQL de la recommandation sélectionnée dans une nouvelle requête : relisez-la, adaptez-la, exécutez-la.
- **Ignorer** écarte une recommandation pour ce projet (elle ne reviendra plus) ; *Analyse › Rétablir les recommandations ignorées* les fait revenir.
- Le **score global** part de 100 et perd 15 points par recommandation critique, 7 par importante et 2 par opportunité.

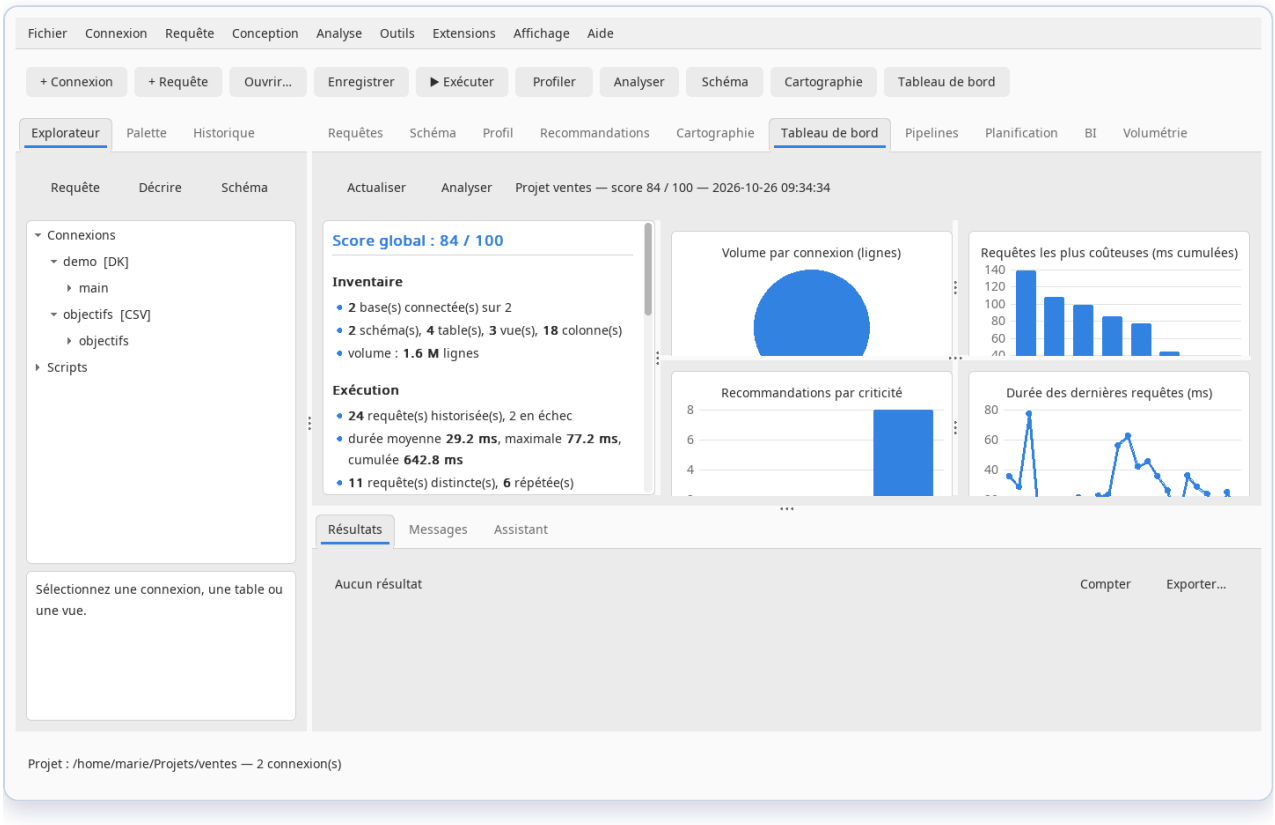
Attention

Une correction modifie souvent la structure (index, table précalculée, clé) : sur une base partagée, faites-la valider, et préférez une connexion en lecture seule pour l'analyse.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

4.7 Le tableau de bord technique

La vue **Tableau de bord** résume la santé du projet : **inventaire** (bases, schémas, tables, vues, colonnes, volume), **exécution** (requêtes historisées, échecs, durées moyenne et maximale, requêtes distinctes et répétées), **recommandations** par niveau avec le gain estimé, et les cinq requêtes les plus coûteuses. Quatre graphiques l'accompagnent : volume par connexion, requêtes les plus coûteuses, recommandations par criticité, durées des trente dernières requêtes.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

CHAPITRE 5 · MODÉLISATION

Modéliser les données

Le concepteur de schéma lit le modèle d'une base ou d'un script, le dessine, le modifie à la souris, mesure l'impact d'un changement et réécrit le tout en SQL.

5.1 Le concepteur · 5.2 Modifier le modèle · 5.3 L'analyse d'impact · 5.4 Modèle → SQL · 5.5 Script → modèle

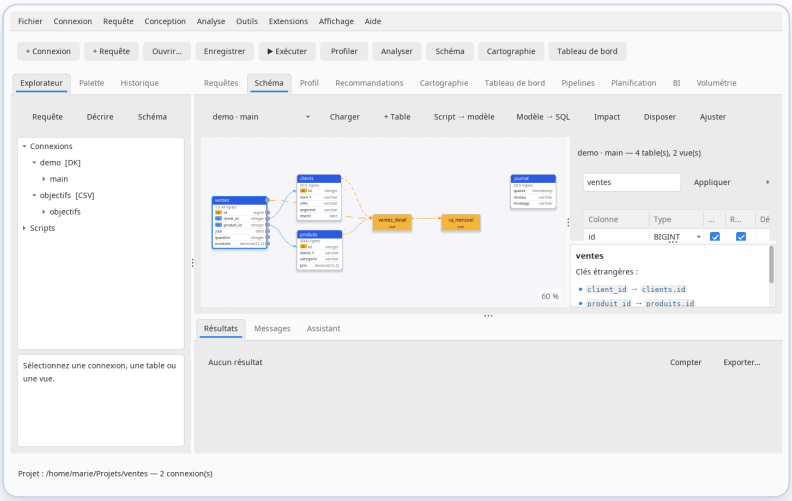


Dans ce chapitre, vous apprendrez à…

- ✔ afficher le modèle d'une base existante
- ✔ mesurer l'impact d'une suppression
- ✔ reconstruire un modèle depuis un script
- ✔ ajouter tables, colonnes et clés étrangères
- ✔ générer le SQL du modèle
- ✔ faire l'aller-retour SQL ↔ modèle

5.1 Le concepteur de schéma

Choisissez une connexion et un schéma dans la liste, puis **Charger** (ou, dans l'explorateur, sélectionnez un schéma et cliquez sur **Schéma**). Le modèle s'affiche : chaque table avec ses colonnes et leur type, les marques **PK** et **FK**, le nombre de lignes ; les clés étrangères relient les colonnes concernées ; les vues, en jaune, sont reliées en pointillé aux tables qu'elles lisent.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

- Glissez une table pour la déplacer : les positions sont mémorisées dans le projet. **Disposer** range tout automatiquement, **Ajuster** cadre le modèle ; la molette zoome.
- Un astérisque après un nom de colonne signale une colonne obligatoire (`NOT NULL`).

5.2 Modifier le modèle

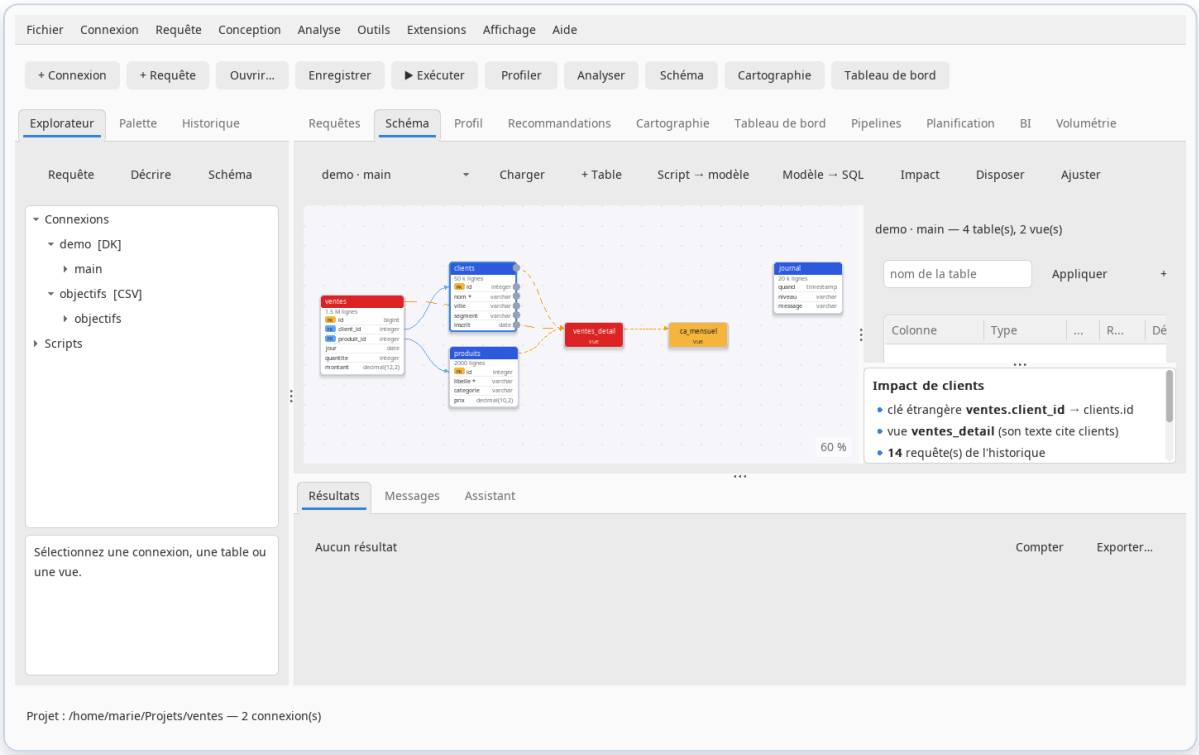
Sélectionnez une table : l'**inspecteur**, à droite, montre son nom et la grille de ses colonnes (nom, type, clé primaire, requise, valeur par défaut). Modifiez-les sur place, ajoutez une colonne avec **+**, puis **Appliquer**. Un renommage de table est propagé aux clés étrangères qui la référencent.

1. **Nouvelle table** : **+ Table**, donnez son nom ; elle reçoit une clé `id`.
2. **Clé étrangère** : tirez la pastille d'une colonne (bord droit de sa ligne) jusqu'à la colonne visée d'une autre table, ou jusqu'à son titre pour viser sa clé primaire ; confirmez.
3. **Suppression** : sélectionnez une table ou un lien et appuyez sur **Suppr** ; pour une table, l'analyse d'impact s'affiche d'abord.

> **Pour bien comprendre**
>
> Le concepteur travaille sur un **modèle**, pas directement sur la base : rien n'est modifié tant que vous n'exécutez pas le SQL généré (5.4). Vous pouvez donc essayer, comparer, et ne garder que ce qui vous convient.

5.3 L'analyse d'impact

Sélectionnez une table (ou une colonne) puis **Impact**. DuckDB Studio recense tout ce qui en dépend : les clés étrangères qui la référencent, les vues dont le texte la cite, les requêtes de l'historique et les scripts `.sql` du projet. Les objets touchés passent en **rouge** dans le diagramme ; la liste détaillée s'affiche sous l'inspecteur.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

5.4 Modèle → SQL

Modèle → SQL écrit le script complet du modèle dans une nouvelle requête : les `CREATE TABLE` dans l'ordre des dépendances (une table après celles qu'elle référence), avec clés primaires et étrangères, puis les index et les vues, chacune après celles qu'elle lit.

GÉNÉRÉ **Le SQL du modèle de démonstration** requeteN.sql

```sql
-- Modèle demo.main généré par DuckDB Studio
CREATE TABLE clients (
  id INTEGER,
  nom VARCHAR NOT NULL,
  ville VARCHAR,
  segment VARCHAR,
  inscrit DATE,
  PRIMARY KEY (id)
);

CREATE TABLE ventes (
  id BIGINT,
  client_id INTEGER,
  produit_id INTEGER,
  jour DATE,
  quantite INTEGER,
  montant DECIMAL(12,2),
  PRIMARY KEY (id),
  FOREIGN KEY (client_id) REFERENCES clients (id),
  FOREIGN KEY (produit_id) REFERENCES produits (id)
);
…
CREATE VIEW ventes_detail AS SELECT … ;
```

5.5 Script → modèle

Dans l'autre sens, **Script → modèle** reconstruit le modèle à partir du script affiché dans l'éditeur. Ses instructions `CREATE` et `ALTER` sont rejouées dans une base temporaire en mémoire, puis le catalogue de cette base est lu : c'est l'analyseur de DuckDB qui interprète le script, avec ses types, ses contraintes et ses vues. Une instruction qui cite un objet défini plus loin est rejouée à la passe suivante ; celles qui échouent sont listées dans Messages.

À retenir

L'aller-retour est fidèle : un modèle écrit en SQL puis relu donne le même modèle. Vous pouvez donc tenir votre modèle dans un script versionné, le dessiner ici pour le modifier, et réécrire le script.

À vous de jouer

Chargez le modèle de `demo.main`, ajoutez une table `retours` (date, motif, quantité) reliée à `ventes`, générez le SQL, puis relisez-le avec **Script → modèle** : la nouvelle table est à sa place, avec sa clé étrangère.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

CHAPITRE 6 · PIPELINES ET PLANIFICATION

Dessiner, traduire et planifier vos traitements

Un pipeline relie des sources, des transformations et des cibles. DuckDB Studio le traduit en SQL lisible, le relit depuis un script, l'exécute, puis le planificateur l'enchaîne avec d'autres travaux.



6.1 Le principe · 6.2 Les étapes · 6.3 Construire
6.4 Valider, exécuter · 6.5 Pipeline → SQL · 6.6 SQL → pipeline
6.7 Le planificateur · 6.8 Les travaux · 6.9 Chaînes et reprises
6.10 Journal et Gantt

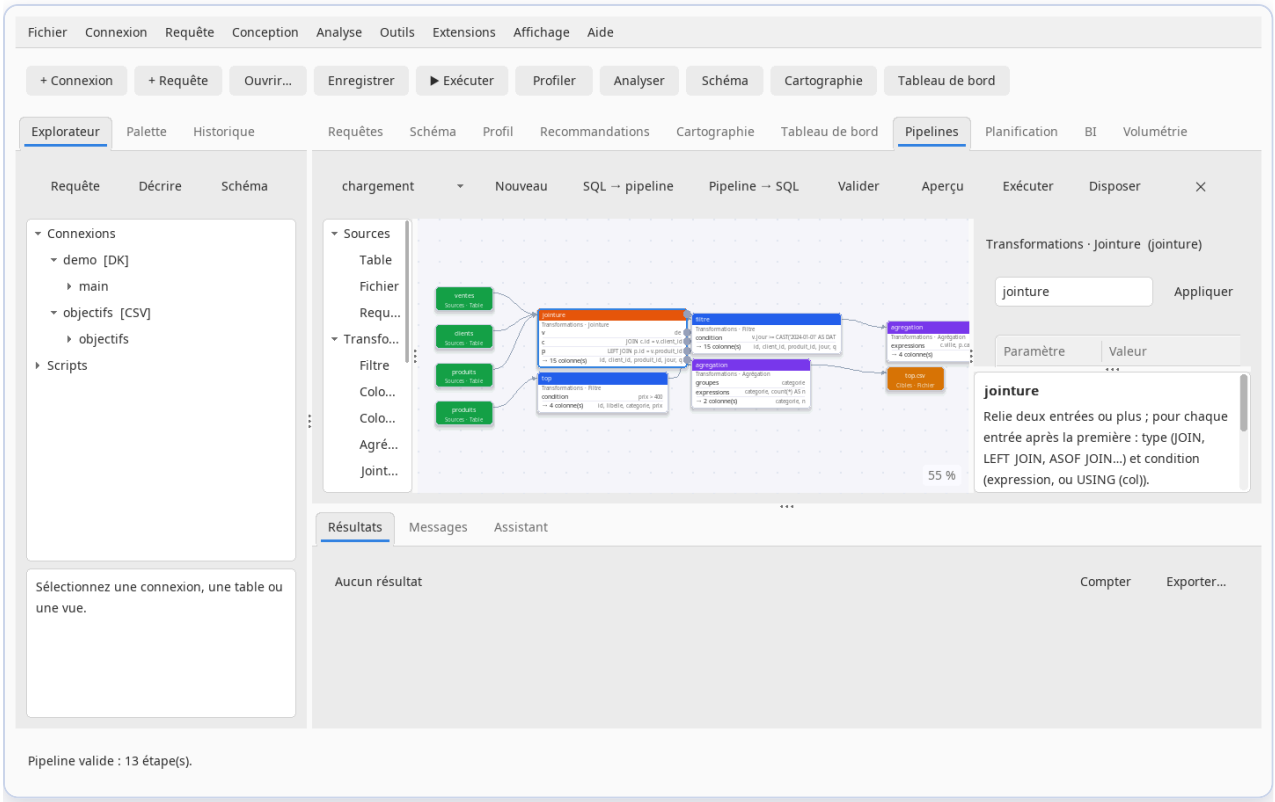
Dans ce chapitre, vous apprendrez à…

- construire un pipeline à la souris
- obtenir le SQL d'un pipeline, et l'inverse
- enchaîner des travaux et gérer les échecs
- vérifier les colonnes et les données de chaque étape
- planifier un travail quotidien ou périodique
- suivre les exécutions sur un diagramme de Gantt

6.1 Le principe

Un **pipeline** est un graphe d'**étapes** reliées par des flèches : les données partent des **sources**, traversent des **transformations** et arrivent dans des **cibles** (une table, un fichier, une vue). La vue **Pipelines** réunit la palette des étapes à gauche, le graphe au centre et l'inspecteur de l'étape sélectionnée à droite.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l



Un pipeline, c'est du SQL

Rien n'est exécuté « à côté » de DuckDB : un pipeline est une autre façon d'écrire des requêtes. Chaque étape correspond à une clause SQL, et DuckDB Studio regroupe les étapes en requêtes lisibles (6.5). Vous gardez toute la puissance — et toute la vitesse — de DuckDB.

6.2 Les étapes

Étape	Paramètres	SQL
Source · Table	table (nom qualifié), alias, colonnes	`FROM demo.ventes AS v`
Source · Fichier	chemin (motifs `*` acceptés), format (auto, csv, parquet, json, excel), alias	`FROM read_parquet('…')`
Source · Requête SQL	requête quelconque	une expression `WITH`
Filtre	condition	`WHERE`, ou `HAVING` après une agrégation
Colonnes	expressions	liste du `SELECT`
Colonne calculée	nom, expression	`SELECT *, expr AS nom`
Agrégation	regrouper par (`ALL` par défaut), expressions	`GROUP BY`
Jointure	alias des entrées, colonnes ; pour chaque entrée après la première : type et condition	`JOIN … ON`, `LEFT JOIN`, `ASOF JOIN`, `USING`
Union	opération	`UNION ALL BY NAME`, `EXCEPT`…

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

Étape	Paramètres	SQL
Tri · Limite	ordre · nombre de lignes	`ORDER BY` · `LIMIT`
Dédoublonnage	clés (vide : toutes les colonnes)	`DISTINCT`, `DISTINCT ON (…)`
SQL libre	requête avec `{entree}`, `{entree2}`…	ce que vous voulez
Cible · Table	table, mode (remplacer, ajouter)	`CREATE OR REPLACE TABLE`, `INSERT INTO … BY NAME`
Cible · Fichier	chemin, format (parquet, csv, json)	`COPY (…) TO`
Cible · Vue	nom de la vue	`CREATE OR REPLACE VIEW`

6.3 Construire un pipeline

1. **Nouveau** crée un pipeline vide et lui donne un nom ; la liste en haut à gauche passe d'un pipeline à l'autre.
2. **Double-cliquez** une étape dans la palette : elle s'ajoute à droite de l'étape sélectionnée et s'y relie si c'est possible. Une source *Table* reprend la table sélectionnée dans l'explorateur.
3. Pour relier deux étapes, **tirez la pastille** du bord droit de la première jusqu'à la seconde. Une étape à entrée unique remplace son entrée ; une jointure ou une union en accepte plusieurs, dans l'ordre des liens.
4. Sélectionnez une étape : réglez son **titre** et ses **paramètres** dans l'inspecteur, puis **Appliquer**. L'aide de l'étape et le SQL qu'elle produit s'affichent dessous.
5. `Suppr` supprime l'étape ou le lien sélectionné ; **Disposer** range le graphe.

Les nœuds résument leurs paramètres (condition, expressions, table…) ; leur couleur indique la famille : vert pour les sources, bleu et violet pour les transformations, orange pour les jointures, ocre pour les cibles.

6.4 Valider, prévisualiser, exécuter

- **Valider** calcule les **colonnes** de chaque étape (DuckDB décrit sa requête) et les affiche dans le nœud (« → 15 colonne(s) ») ; une étape en erreur passe en **rouge**, son message s'affiche dans l'inspecteur.
- **Aperçu** montre les 100 premières lignes produites par l'étape sélectionnée : le moyen le plus sûr de vérifier un filtre ou une jointure avant d'aller plus loin.
- **Exécuter** exécute les cibles dans l'ordre des dépendances ; chaque instruction est historisée, les catalogues sont relus.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

6.5 Pipeline → SQL

Pipeline → SQL écrit le script du pipeline dans une nouvelle requête. Les étapes d'une même chaîne sont **fusionnées** en un seul bloc `SELECT … FROM … WHERE … GROUP BY … HAVING … ORDER BY … LIMIT`, dans l'ordre des clauses SQL ; une étape partagée par plusieurs branches, ou qui ne peut pas prolonger le bloc (un filtre après un tri, par exemple), devient une expression de table commune (`WITH`) qui porte son nom.

GÉNÉRÉ **Le SQL du pipeline « chargement »** requeteN.sql

```sql
-- Pipeline « chargement » (DuckDB Studio)

-- ca_ville
CREATE OR REPLACE TABLE ca_ville AS
SELECT c.ville, p.categorie, sum(v.montant) AS ca, count(*) AS nb
FROM demo.ventes AS v
  JOIN demo.clients AS c ON c.id = v.client_id
  LEFT JOIN demo.produits AS p ON p.id = v.produit_id
WHERE v.jour >= CAST('2024-01-01' AS DATE)
GROUP BY ALL
HAVING sum(v.montant) > 1000
ORDER BY ca DESC;

-- top.csv
COPY (
  SELECT categorie, count(*) AS n
  FROM demo.produits
  WHERE prix > 400
  GROUP BY categorie
) TO '/home/marie/Projets/ventes/top.csv' (FORMAT csv);
```

6.6 SQL → pipeline

SQL → pipeline fait l'inverse : il construit un pipeline à partir du script de l'éditeur (ou de la sélection). Chaque requête est analysée par **DuckDB lui-même** ; les jointures, filtres, agrégations, `HAVING`, tris, limites, `DISTINCT`, unions, sous-requêtes et expressions de table deviennent des étapes, et `CREATE TABLE … AS`, `INSERT INTO`, `COPY … TO` et `CREATE VIEW` des cibles. Le script ci-dessous a produit le pipeline de la page précédente.

EXEMPLE **Le script d'origine** chargement.sql

```sql
-- ventes par ville et catégorie, clients actifs
CREATE OR REPLACE TABLE ca_ville AS
SELECT c.ville, p.categorie, sum(v.montant) AS ca, count(*) AS nb
FROM demo.ventes v
  JOIN demo.clients c ON c.id = v.client_id
  LEFT JOIN demo.produits p ON p.id = v.produit_id
WHERE v.jour >= DATE '2024-01-01'
GROUP BY ALL
HAVING sum(v.montant) > 1000
ORDER BY ca DESC;

```


© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

EXEMPLE **Le script d'origine (suite)** chargement.sql

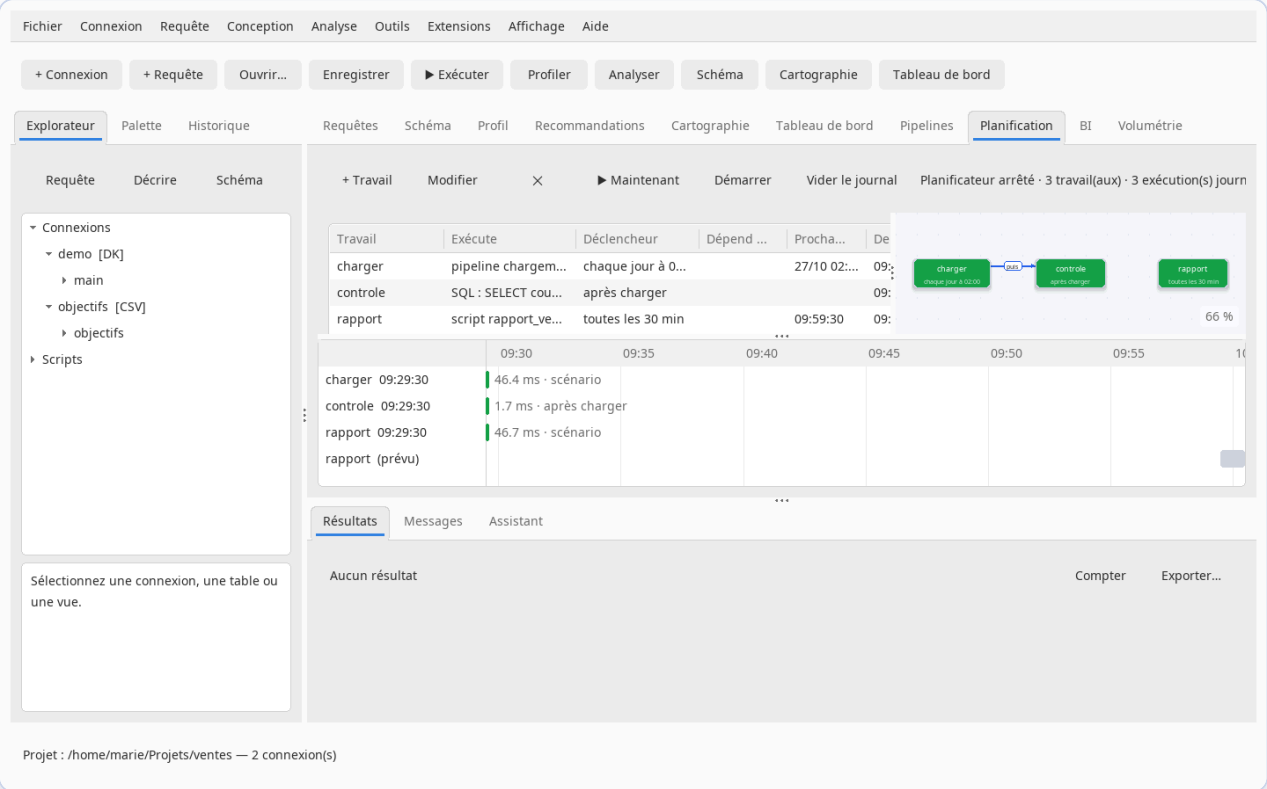
```sql
COPY (
  WITH top AS (SELECT * FROM demo.produits WHERE prix > 400)
  SELECT categorie, count(*) AS n FROM top GROUP BY categorie
) TO 'top.csv' (FORMAT csv);
```

À retenir

L'aller-retour est exact : SQL → pipeline → SQL redonne le même texte, mis en forme. Chaque expression est réécrite par DuckDB, quelles que soient les fonctions, les `CASE` ou les sous-requêtes qu'elle contient. Ce qui sort du modèle en étapes (`QUALIFY`, `SAMPLE`, requête sans `FROM`) est gardé tel quel dans une étape *Requête SQL*.

6.7 Le planificateur

La vue **Planification** automatise l'exécution : des **travaux** exécutent un pipeline, une requête SQL ou un script du projet, à la main, à intervalle régulier, chaque jour à heure fixe, ou à la suite d'autres travaux.



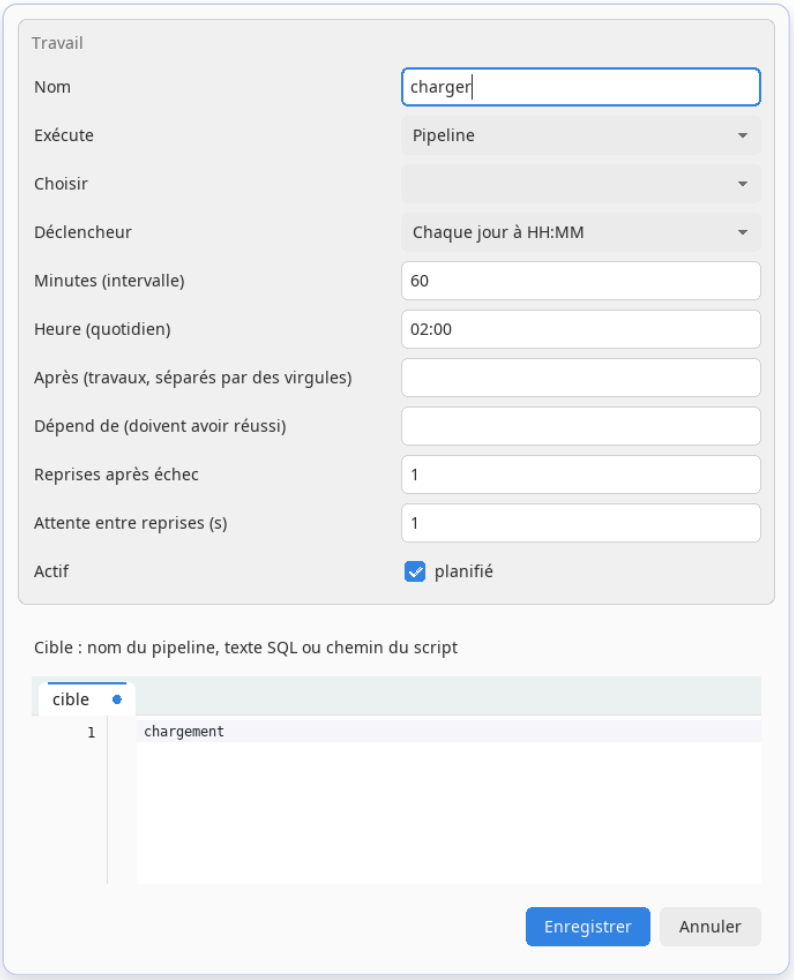
- En haut à gauche, le **tableau des travaux** : ce qu'ils exécutent, leur déclencheur, leurs dépendances, leurs prochaine et dernière exécutions et leur état.
- En haut à droite, le **graphe** des enchaînements (« puis ») et des dépendances (« requiert ») ; la couleur donne l'état : vert réussi, rouge échec, orange bloqué, bleu en cours.
- En bas, le **diagramme de Gantt** des exécutions (6.10).

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

Démarrer lance le planificateur : il examine les échéances toutes les vingt secondes, tant que DuckDB Studio est ouvert. Son état est mémorisé dans le projet. ▶ **Maintenant** exécute le travail sélectionné tout de suite, avec sa chaîne.

6.8 Les travaux

+ Travail (ou un double-clic sur un travail pour le modifier) ouvre la fiche du travail.



Champ	Rôle
Exécute · Choisir	pipeline, requête SQL ou script `.sql` ; la liste propose les pipelines, les scripts ou des modèles de requêtes
Cible	nom du pipeline, texte SQL (plusieurs instructions possibles) ou chemin du script
Déclencheur	manuel ; toutes les N minutes ; chaque jour à HH:MM ; après d'autres travaux
Après	travaux après lesquels exécuter celui-ci (séparés par des virgules)
Dépend de	travaux dont la dernière exécution doit avoir réussi
Reprises · Attente	nombre de nouvelles tentatives après un échec, et délai entre elles
Actif	un travail inactif n'est jamais déclenché par le planificateur

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

EXEMPLE Un contrôle de qualité, en travail SQL — travail « controle »

```sql
-- échoue (et déclenche les reprises) si la table est vide
SELECT CASE WHEN count(*) = 0 THEN error('ca_ville est vide') END
FROM ca_ville;
```

6.9 Chaînes, dépendances et reprises

- **Chaîne** : un travail déclenché « après » d'autres s'exécute dès qu'ils ont tous réussi. Sur la capture, `controle` suit `charger`.
- **Dépendance** : un travail planifié qui « dépend de » travaux en échec (ou jamais exécutés) ne s'exécute pas ; il passe à l'état **bloqué** et attend sa prochaine échéance.
- **Reprise** : après un échec, le travail est relancé au bout de l'attente indiquée, autant de fois que le nombre de reprises le permet ; le symbole ↻ le signale dans le tableau. Le dernier échec s'affiche dans Messages.

> **Attention**
> Le planificateur fonctionne tant que DuckDB Studio est ouvert. Pour des traitements de nuit sur un serveur, gardez une session ouverte, ou faites exécuter le SQL de vos pipelines (6.5) par le planificateur du système.

6.10 Le journal et le Gantt

Chaque exécution est journalisée dans le projet (les 500 dernières) : travail, début, durée, réussite, message, tentative et motif (planifié, manuel, reprise, après tel travail). Le diagramme de Gantt affiche les quarante dernières, sur un axe horaire : **vert** réussie, **rouge** échec, **gris** prévue dans les six prochaines heures. **Vider le journal** l'efface.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

CHAPITRE 7 · BI ET ASSISTANT

Restituer et se faire aider

Des tableaux de bord construits sur vos requêtes, en deux clics ; un assistant qui explique les plans et les erreurs, réécrit les requêtes, écrit le SQL d'une question et conçoit des modèles.



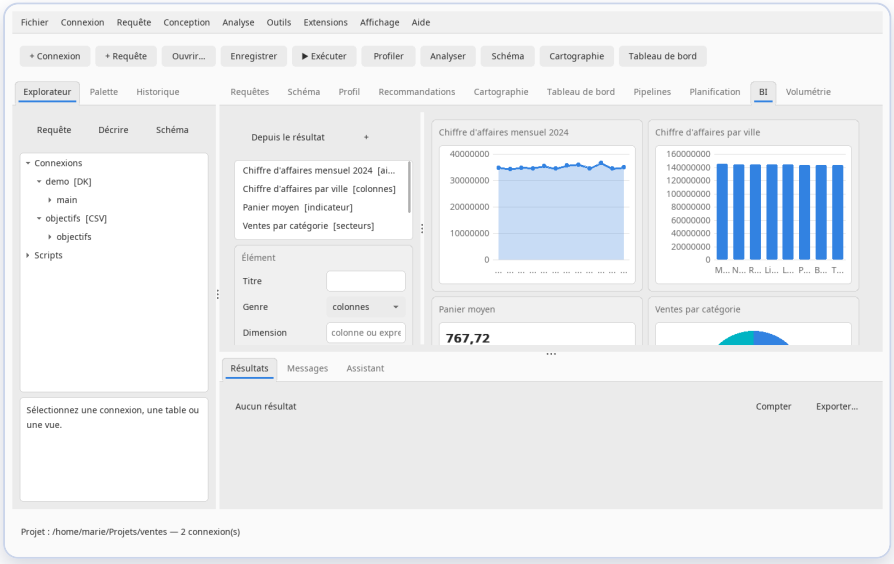
7.1 Le tableau de bord BI
7.2 Créer un élément
7.3 Genres et agrégats
7.4 L'assistant
7.5 Les actions
7.6 Les moteurs
7.7 Générer un modèle
7.8 Confidentialité

Dans ce chapitre, vous apprendrez à…

- ✔ transformer un résultat en graphique
- ✔ composer un tableau de bord
- ✔ brancher un modèle local ou un serveur
- ✔ régler dimension, mesures et agrégat
- ✔ demander une explication ou une réécriture
- ✔ générer un modèle de données

7.1 Le tableau de bord BI

La vue **BI** présente des **éléments** — graphiques, indicateurs, mosaïques, tableaux — chacun calculé par une requête. Ils sont enregistrés dans le projet et recalculés à l'ouverture et à chaque clic sur ↻.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

7.2 Créer un élément

1. Exécutez une requête dans l'éditeur.
2. Dans la vue BI, cliquez sur **Depuis le résultat** : un élément est créé sur cette requête, avec pour **dimension** sa première colonne texte et pour **mesure** sa première colonne numérique.
3. Réglez le **titre**, le **genre**, la dimension, les **mesures** (plusieurs, séparées par des virgules : une série par mesure), l'**agrégat** et la **limite**, puis **Appliquer**.
4. **Colonnes** liste les colonnes de la requête source (**#** marque les numériques) ; ▲ avance l'élément, ✕ le supprime.

La requête affichée regroupe la requête source par la dimension :

GÉNÉRÉ **Ce que calcule l'élément « Chiffre d'affaires par ville »** BI

```
-- la requête source est regroupée par la dimension, une colonne par mesure
SELECT ville AS d, sum(ca) AS m1
FROM (
  SELECT c.ville, count(*) AS ventes, sum(v.montant) AS ca
  FROM demo.ventes v JOIN demo.clients c ON c.id = v.client_id
  GROUP BY c.ville
) AS _source
GROUP BY 1 ORDER BY 2 DESC LIMIT 12
```

7.3 Genres et agrégats

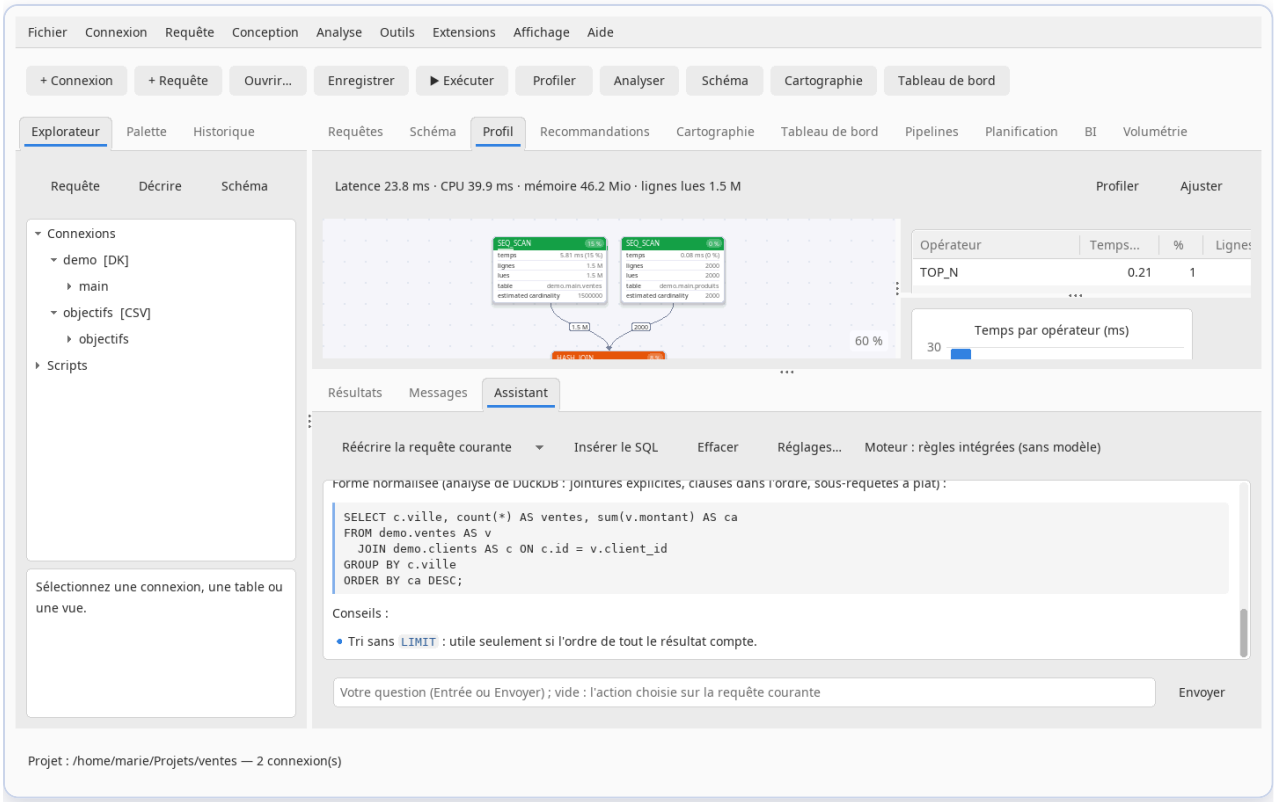
Genre	Usage
colonnes	comparer des catégories (triées de la plus grande à la plus petite)
lignes · aires	une évolution dans le temps (triée par dimension)
secteurs	une répartition en quelques parts
mosaique	une répartition en surfaces proportionnelles, lisible avec beaucoup de catégories
indicateur	une seule valeur, en grand : total, moyenne, nombre
tableau	les premières lignes de la requête source, valeurs mises en forme

Agrégats : **somme**, **moyenne**, **nombre**, **minimum**, **maximum**, **distincts** (nombre de valeurs différentes) et **aucun** (valeurs telles quelles). La **limite** borne le nombre de catégories.

7.4 L'assistant

L'onglet **Assistant**, en bas de la fenêtre, est une conversation : choisissez une **action**, écrivez éventuellement une question, puis **Envoyer** (ou Entrée). La réponse s'affiche en Markdown ; **Insérer le SQL** copie le dernier bloc SQL proposé dans une nouvelle requête, **Effacer** vide la conversation.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l



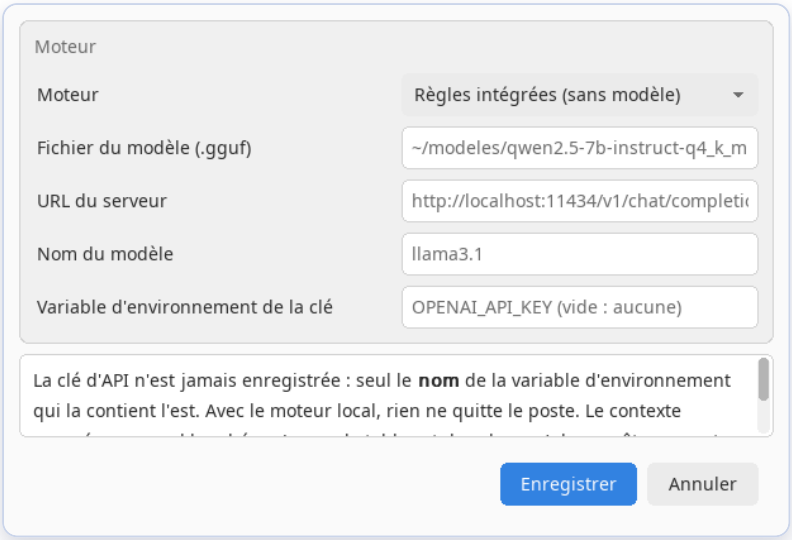
7.5 Les actions

Action	Ce que l'assistant reçoit et rend
Question libre	votre question, le schéma et la requête courante ; une réponse argumentée
Écrire le SQL d'une question	« combien de ventes par ville en 2024 ? » ; une requête sur vos tables
Expliquer le dernier plan	le profil (`F6`) ; les étapes coûteuses expliquées, les optimisations
Réécrire la requête courante	la requête sous le curseur ; une version plus lisible et plus rapide
Expliquer la dernière erreur	la requête et le message de DuckDB ; la cause et la correction
Générer un modèle de données	une description ; un script `CREATE TABLE` , ouvert aussitôt dans le concepteur de schéma

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

7.6 Les moteurs

Réglages… choisit le moteur de l'assistant :



Moteur	Fonctionnement
Règles intégrées	sans modèle de langage, toujours disponible : plan commenté opérateur par opérateur avec les recommandations, forme normalisée de la requête (jointures explicites, clauses dans l'ordre) et conseils, diagnostic des erreurs courantes avec les noms proches du catalogue, requête de départ, modèle déduit d'une description structurée
Modèle local GGUF	un fichier `.gguf` (Llama 3, Mistral, Qwen…) chargé par le paquet `llm` de Lexis+ : rien ne quitte le poste ; la réponse s'affiche au fil de la génération
Serveur HTTP	une API compatible OpenAI (`/v1/chat/completions`) : Ollama, llama.cpp, LM Studio, vLLM ou un fournisseur en ligne

GÉNÉRÉ **Ce qu'envoie le moteur HTTP** requête HTTP

```
POST http://localhost:11434/v1/chat/completions
Authorization: Bearer $OPENAI_API_KEY        (si une variable est indiquée)
{"model": "llama3.1", "temperature": 0.2,
 "messages": [{"role": "system", "content": "Tu es l'assistant de DuckDB Studio…"},
              {"role": "user", "content": "Schéma disponible : demo.main.clients(id integer PK,
…)…"}]}
```

Si le moteur choisi échoue (serveur arrêté, fichier introuvable), la réponse le signale et les règles intégrées prennent le relais.

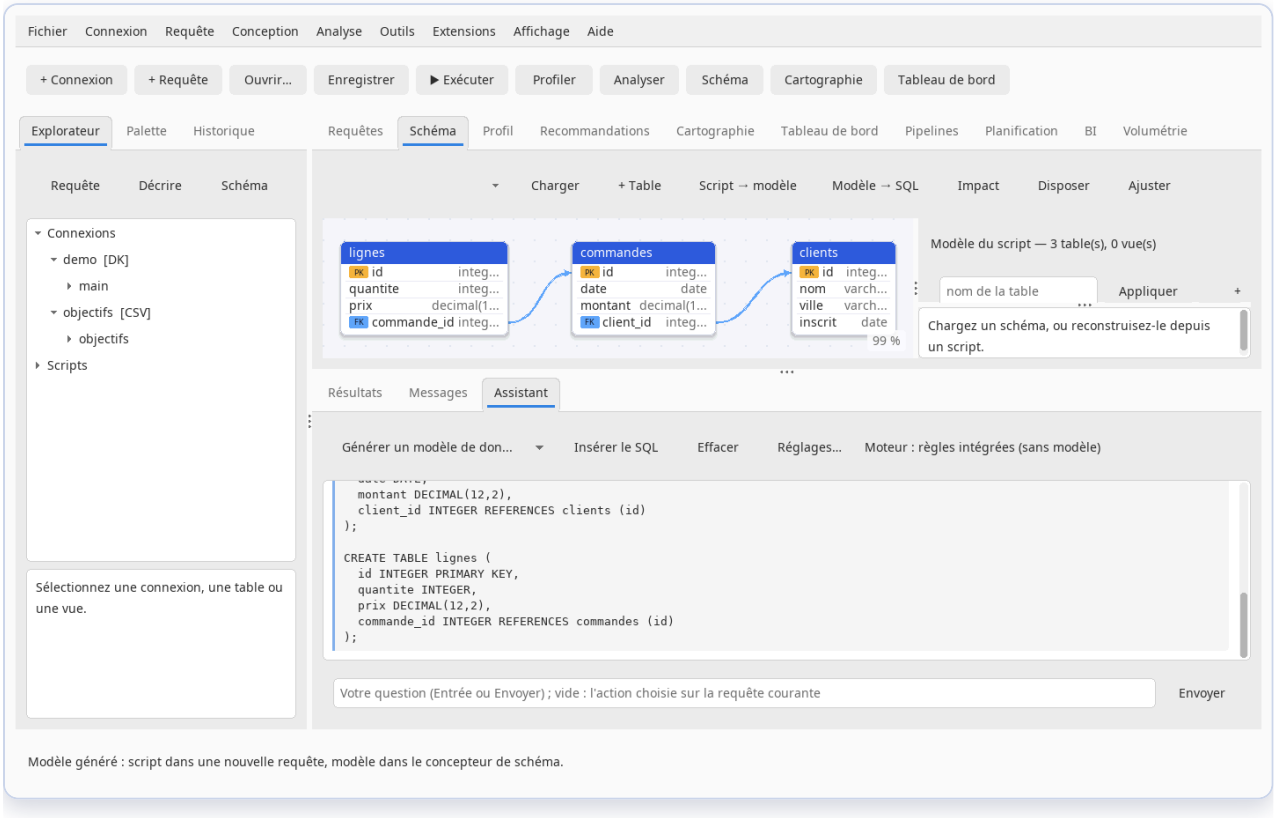
7.7 Générer un modèle

Avec un modèle de langage, décrivez simplement le besoin (« une médiathèque : adhérents, ouvrages, exemplaires, prêts »). Avec les règles intégrées, utilisez une description structurée : une ligne par table, ses colonnes avec leur type éventuel, et `->` vers les tables référencées.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

SYNTAXE **Description structurée** question

```
clients : nom, ville, inscrit date
commandes : date date, montant decimal -> clients
lignes : quantite entier, prix decimal -> commandes
```



Chaque table reçoit une clé `id`, chaque `->` une clé étrangère ; les types sont déduits des noms (`date`, `montant`, `quantite`…) quand ils ne sont pas donnés. Le script s'ouvre dans une nouvelle requête et le modèle dans le concepteur de schéma (chapitre 5), prêt à être retouché.

7.8 Confidentialité

À retenir

- Le contexte transmis à un modèle comprend les **noms** de tables et de colonnes (avec types, clés et volumes), la requête courante, le dernier plan et la dernière erreur : **jamais les données**.
- La **clé d'API** n'est jamais enregistrée : seul le nom de la variable d'environnement qui la contient l'est (par exemple `OPENAI_API_KEY`).
- Avec le moteur local ou les règles intégrées, rien ne quitte le poste.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

CHAPITRE 8 · EXTENSIONS

Étendre DuckDB Studio

Vos règles d'expertise, vos onglets, vos commandes et vos connecteurs : quelques lignes de Lexis+ ou de JSON dans le projet, sans modifier l'atelier.

8.1 Le principe · 8.2 Charger et faire confiance · 8.3 Écrire une extension · 8.4 Le gestionnaire · 8.5 Un connecteur



Dans ce chapitre, vous apprendrez à…

- ✔ créer une extension à partir du modèle
- ✔ ajouter un onglet et des commandes
- ✔ décrire un nouveau connecteur en JSON
- ✔ ajouter une règle au moteur d'expertise
- ✔ recharger une extension modifiée
- ✔ savoir ce qu'une extension peut faire

8.1 Le principe

Une **extension** est un fichier `.prg` du dossier `extensions/` du projet. Elle ne contient que des procédures et des fonctions Lexis+. DuckDB Studio les charge dans son propre programme, puis appelle chaque fonction `EXTENSION_xxx()` qu'il y trouve : elle déclare ce que l'extension apporte.

SYNTAXE **La déclaration d'une extension** — extensions/maison.prg

```
fonction EXTENSION_MAISON()
retourner {nom = "Règles maison", version = "1.0", description = "…", ;
           regles = @MAISON_REGLES, ;              && fonction(contexte) -> liste de RECO(…)
           vue = @MAISON_VUE, titrevue = "Volumétrie", ;   && procédure(page) : un onglet
           menus = {{libelle = "Compter les lignes", action = @MAISON_COMPTER}}}
```


© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

Champ	Rôle
nom, version, description	affichés dans le gestionnaire
regles	fonction appelée par **Analyser** : reçoit le contexte `{objets, historique, plan, connexions}`, renvoie une liste de recommandations
vue, titrevue	procédure qui construit un onglet (elle reçoit la page) et son titre
menus	entrées du menu **Extensions** : `{libelle, action}`

8.2 Charger et faire confiance

Une extension est du code qui s'exécute avec les droits de DuckDB Studio : elle peut lire et écrire des fichiers, interroger vos bases, accéder au réseau. C'est pourquoi un projet qui contient des extensions n'est pas chargé d'office : à l'ouverture, la barre d'état et Messages signalent les extensions en attente. *Extensions › Recharger les extensions* demande alors votre accord ; une fois accordé, le projet est marqué **de confiance** et ses extensions se chargent ensuite automatiquement.

> **Attention**
> Ne faites confiance qu'aux projets dont vous connaissez l'origine, comme pour une macro de tableur. Le gestionnaire (8.4) liste chaque extension et son fichier : ouvrez-le et lisez-le avant de l'accepter.

8.3 Écrire une extension

Extensions › Nouvelle extension… demande un nom, crée le fichier à partir d'un modèle commenté (`modeles/extension.txt`) et l'ouvre dans l'éditeur. L'exemple ajoute deux règles de gouvernance, un onglet **Volumétrie** (une mosaïque des tables proportionnelle à leur nombre de lignes) et une commande qui compte exactement les lignes de chaque table.

EXEMPLE **L'extension d'exemple** — extensions/regles_maison.prg

```
fonction EXTENSION_REGLES_MAISON()
retourner {nom = "Règles maison", version = "1.0", description = "Règles maison, volumétrie en mosaïque", ;
          regles = @REGLES_MAISON_REGLES, vue = @REGLES_MAISON_VUE, titrevue = "Volumétrie", ;
          menus = {{libelle = "Compter exactement les lignes des tables", action = @REGLES_MAISON_COMPTER}}}

fonction REGLES_MAISON_REGLES(ctx)
  locale l = {}
  pour chaque o dans ctx.objets
    locale n = minuscule(o.nom)
    && tables de travail oubliées
    si o.genre = "table" et (commencepar(n, "tmp_") ou commencepar(n, "test") ou finitpar(n, "_old") ou finitpar(n, "_bak"))
      inserer(l, RECO("Opportunité", "Gouvernance", "Table de travail « " + QUALIFIE(o) + " »", ;
```


© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

EXEMPLE L'extension d'exemple (suite) — extensions/regles_maison.prg

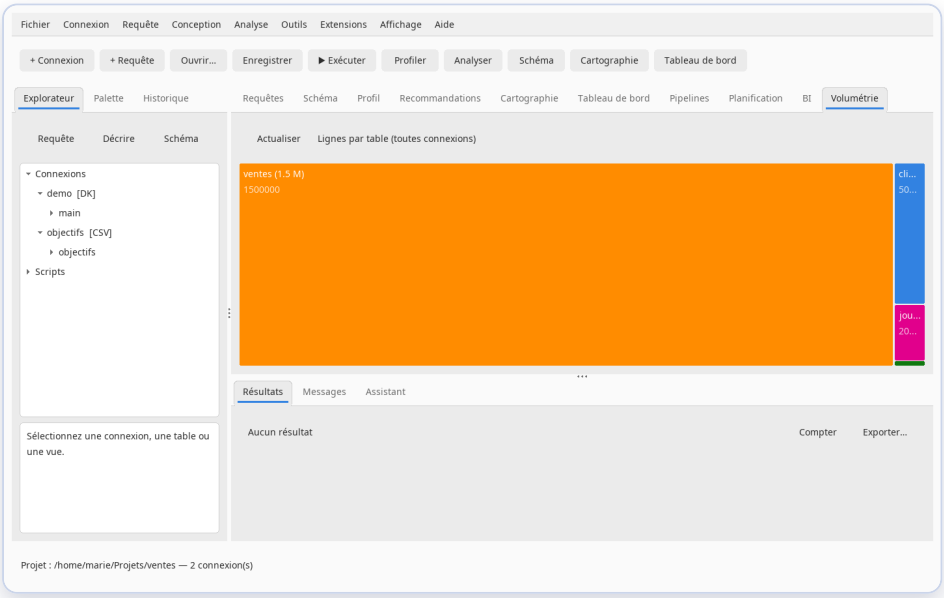
```
        "Son nom indique une table temporaire ou de sauvegarde : elle occupe de la place et brouille le catalogue.", ;
        "stockage", "La supprimer si elle n'est plus utile.", "DROP TABLE " + QUALIFIE(o) + ";"))
    finsi
    && colonnes *_id sans clé étrangère
    si o.genre = "table" et o.objet.colonnes <> nul
      pour chaque c dans o.objet.colonnes
        si finitpar(minuscule(c.nom), "_id") et #o.objet.fk = 0
          inserer(l, RECO("Opportunité", "Modélisation", "Colonne « " + o.nom + "." + c.nom + " » sans clé étrangère", ;
            "Son nom suggère une référence vers une autre table, mais aucune contrainte ne la déclare : " + ;
            "le concepteur de schéma et l'analyse d'impact l'ignorent.", "intégrité", ;
            "Déclarer la clé étrangère (concepteur de schéma : tirer la colonne vers la table cible).", nul))
        finsi
      suivant
    finsi
  suivant
retourner l

&& … puis REGLES_MAISON_VUE (l'onglet Volumétrie), REGLES_MAISON_ACTUALISER
&& et REGLES_MAISON_COMPTER (la commande du menu) : voir le fichier complet.
```

Une recommandation se construit avec la fonction `RECO` de DuckDB Studio ; toutes ses autres procédures sont disponibles : `requete`, `JOURNAL`, `STATUT`, `TOUS_LES_OBJETS`, `QUALIFIE`, `LIGNES_TEXTE`… ainsi que les paquets de Lexis+ (`interface`, `fichier`, `http`…).

SYNTAXE Une recommandation — expertise.prg

```
RECO(niveau, domaine, probleme, justification, gain, correction, sql)
&& niveau : "Critique", "Important" ou "Opportunité"
&& sql    : correction proposée (nul : aucune)
```



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

> **Astuce**
>
> Après une modification, *Extensions › Recharger les extensions* remplace les procédures de l'extension sans redémarrer l'atelier. Un nom de procédure déjà utilisé par DuckDB Studio ou par une autre extension est refusé : préfixez les vôtres (`MAISON_…`).

8.4 Le gestionnaire d'extensions

Extensions › Gestionnaire d'extensions… liste les extensions du projet : nom, version, apports (règles, onglet, menus), fichier et état (chargée, non chargée, erreur avec son message). **Ouvrir** affiche le fichier sélectionné dans l'éditeur, **Recharger** les recharge toutes.



8.5 Ajouter un connecteur

Les connecteurs sont des fichiers JSON : ceux de DuckDB Studio sont dans `connecteurs/` à côté du programme ; déposez les vôtres dans le dossier `connecteurs/` du projet (ils sont lus à l'ouverture du projet, s'ajoutent à la liste ou remplacent un connecteur du même type). Voici celui de Quack :

SYNTAXE **Le connecteur Quack** — connecteurs/quack.json

```
{
  "type": "quack",
  "libelle": "Quack (serveur DuckDB distant)",
  "categorie": "Bases",
  "sigle": "QK",
  "couleur": "#FB923C",
  "mode": "catalogue",
  "version_min": "2.0",
  "extension": "quack",
  "parametres": [
    {
      "nom": "hote",
      "libelle": "Hôte",
      "defaut": "localhost"
    },
```


© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

SYNTAXE **Le connecteur Quack (suite)** connecteurs/quack.json

```
    {
      "nom": "port",
      "libelle": "Port",
      "defaut": "9494"
    },
    {
      "nom": "jeton",
      "libelle": "Jeton",
      "defaut": "",
      "secret": true
    }
  ],
  "options": [
    {
      "nom": "sanstls",
      "libelle": "Sans TLS (DISABLE_SSL)",
      "defaut": false
    }
  ],
  "attacher": [
    "ATTACH 'quack:{hote}:{port}' AS {alias} (TOKEN '{jeton}'{si_sanstls:, DISABLE_SSL true})"
  ]
}
```

Champ	Rôle
type, libelle, categorie	identifiant, nom affiché, rubrique de la liste
sigle, couleur	marque dans l'explorateur et la cartographie
mode	`catalogue` (ATTACH puis introspection), `fichiers` (une vue par fichier), `requete` (fonctions de table)
extension, installer	extension DuckDB à charger ; commande d'installation particulière
parametres	`{nom, libelle, defaut, secret, fichier, dossier}` : champs de la boîte
options	`{nom, libelle, defaut}` : cases à cocher
attacher	instructions SQL : `{param}` devient la valeur, `{alias}` le nom SQL, `{si_option:texte}` le texte si l'option est cochée
lecteur	mode fichiers : fonction de lecture, `{chemin}` désignant le fichier
version_min, aide	version minimale de DuckDB ; texte d'aide

À vous de jouer

Copiez `connecteurs/csv.json` dans le dossier `connecteurs` de votre projet sous le nom `tsv.json`, changez `type`, `libelle`, le motif (`*.tsv`) et le lecteur (`"read_csv('{chemin}', delim = '\\t')"`), puis rouvrez le projet : le type « Fichiers TSV » apparaît dans la boîte de connexion.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

DS ANNEXE A

Référence complète

Les boutons de chaque vue, les fichiers du projet et des réglages, les champs des recommandations et des extensions, et les sources de l'atelier pour qui veut l'adapter.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l



Référence · Vues et boutons

Les commandes de chaque vue, dans l'ordre de leur barre d'outils.

Explorateur

Bouton	Effet
`Requête · Décrire · Schéma · ↻`	SELECT … LIMIT 100 · SUMMARIZE · concepteur de schéma · relire les catalogues

Résultats

Bouton	Effet
`Compter`	nombre exact de lignes de la dernière requête
`Exporter…`	COPY du résultat complet en Parquet, CSV ou JSON

Profil · Recommandations · Tableau de bord

Bouton	Effet
`Profiler · Ajuster`	EXPLAIN ANALYZE de la requête sous le curseur · cadrer le plan
`Analyser`	règles sur le plan, les catalogues, l'historique, les empreintes et les extensions
`Générer la correction · Ignorer`	correction SQL dans une nouvelle requête · écarter pour ce projet
`Actualiser`	recalcule les indicateurs du tableau de bord technique

Schéma

Bouton	Effet
`Charger`	modèle de la connexion et du schéma choisis
`+ Table`	nouvelle table (clé id)
`Script → modèle · Modèle → SQL`	relire le script de l'éditeur · écrire le SQL du modèle
`Impact`	dépendances de la table ou de la colonne sélectionnée
`Disposer · Ajuster`	disposition automatique · cadrer
`Appliquer · +`	inspecteur : enregistrer nom et colonnes · ajouter une colonne

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

Cartographie

Commande	Effet
`filtre · recherche`	une connexion ou toutes · tables et vues dont le nom contient le texte
`Tables seules · Vues · Grouper par schéma`	contenu de la carte
`Afficher · Ajuster`	recalculer · cadrer

Pipelines

Bouton	Effet
`Nouveau · ×`	créer · supprimer le pipeline
`SQL → pipeline · Pipeline → SQL`	depuis le script de l'éditeur · vers une nouvelle requête
`Valider · Aperçu · Exécuter`	colonnes et erreurs · 100 lignes de l'étape · exécuter les cibles
`Disposer · Appliquer`	ranger le graphe · enregistrer les paramètres de l'étape

Planification

Bouton	Effet
`+ Travail · Modifier · ×`	fiche d'un travail (aussi : double-clic)
`▶ Maintenant`	exécuter le travail sélectionné et sa chaîne
`Démarrer / Arrêter`	planificateur (échéances examinées toutes les 20 s)
`Vider le journal`	effacer les exécutions mémorisées

BI · Assistant

Bouton	Effet
`Depuis le résultat · + · × · ↻`	élément sur la dernière requête · élément vide · supprimer · tout recalculer
`Colonnes · Appliquer · ▲`	colonnes de la source · enregistrer l'élément · l'avancer
`Envoyer · Insérer le SQL`	poser la question · dernier bloc SQL dans une nouvelle requête
`Effacer · Réglages…`	vider la conversation · choisir le moteur

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l



Référence · Fichiers

Le projet, les réglages personnels et les sources de l'atelier.

duckstudio.json (dossier du projet)

Clé	Contenu
CONNEXIONS	{nom, description, type, parametres, options} ; jamais les secrets
HISTORIQUE	500 dernières requêtes : {sql, quand, duree (ms), lignes, ok, message}
POSITIONS · IGNOREES	positions des tables du concepteur · recommandations ignorées
PIPELINES	{nom, etapes {id, genre, titre, params, x, y}, liens {de, a}}
TRAVAUX · EXECUTIONS	travaux planifiés · journal (500 dernières exécutions)
PLANIFICATEUR	vrai si le planificateur est démarré
BI	{titre, genre, sql, dimension, mesures, agregat, limite}
EXTENSIONS_CONFIANCE	vrai : les extensions du projet sont chargées sans demande

~/.duckdbstudio.json (réglages personnels)

Clé	Contenu
PROJET · SOMBRE · POLICE · LIMITE	dernier projet · thème · taille de police · lignes affichées (1000)
IA_MOTEUR	regles, local ou http
IA_MODELE	chemin du fichier .gguf (moteur local)
IA_URL · IA_NOM	adresse /v1/chat/completions · nom du modèle (moteur http)
IA_CLE_ENV	nom de la variable d'environnement qui contient la clé

Recommandations et extensions

Élément	Contenu
RECO(niveau, domaine, probleme, justification, gain, correction, sql)	niveau : Critique, Important, Opportunité ; sql : correction ou nul
r.actuel · r.apres	durées en secondes avant / après (retour sur investissement affiché)
contexte.objets	{cnx, alias, schema, nom, genre, objet, fichiers} ; objet : {colonnes, pk, fk, index, lignes}
contexte.historique · plan · connexions	historique du projet · dernier plan profilé · connexions
EXTENSION_xxx()	{nom, version, description, regles, vue, titrevue, menus}

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

Sources de l'atelier (exemples/duckdbstudio)

Fichier	Rôle
`principal.prg · menus.prg`	fenêtre, barre d'outils, réglages, démarrage · menus et raccourcis
`projet.prg · demo.prg`	projet, historique, scripts · base de démonstration
`connecteurs.prg · connexions.prg`	greffons, attachement, catalogue · boîte de connexion
`explorateur.prg · carto.prg`	arbre, fiches, palette · cartographie
`requetes.prg · sqlast.prg`	éditeur, exécution, résultats · arbre syntaxique de DuckDB
`profil.prg · expertise.prg · mutualisation.prg`	profil · règles · empreintes
`tableau.prg · schema.prg`	tableau de bord technique · concepteur de schéma
`pipelines.prg · planificateur.prg`	pipelines · travaux et Gantt
`bi.prg · assistant.prg · extensions.prg`	BI · assistant · extensions
`connecteurs/*.json · modeles/extension.txt`	les douze connecteurs · modèle d'extension

Les briques de Lexis+ utilisées

DuckDB Studio montre ce que Lexis+ sait faire : la fonction `requete` (DuckDB), l'éditeur `ui.editeur` en mode SQL, le canevas `ui.diagramme` (schéma, plans, pipelines, cartographie), `ui.gantt`, `ui.minuterie` (planificateur), le paquet `llm` (assistant local), `http` (assistant distant) et `lexis.charger` (extensions). Elles sont décrites dans la documentation de wdlexis+.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l

DS ANNEXE B

Aides-mémoires

L'atelier sur deux pages : interroger et optimiser, puis concevoir et automatiser. Les gestes, les touches, les enchaînements et les pièges. À imprimer et à garder sous la main.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de l



Aide-mémoire · Interroger et optimiser

Se connecter, écrire, exécuter, mesurer, corriger.

Voir aussi : Chapitres 2, 3 et 4 · Assistant (7.4)

F5 · F6
exécuter · profiler

ENCHAÎNEMENTS : + Connexion → Tester → Enregistrer → explorateur : Requête / Décrire — F5 → F6 → Analyser → Générer la correction → F6

Requêtes types

```
-- objet : connexion.schéma.table
SELECT * FROM demo.ventes LIMIT 100;
SUMMARIZE demo.clients;
SELECT ville, sum(montant) AS ca
FROM demo.ventes_detail GROUP BY ALL;
-- fichiers sans connexion
SELECT * FROM read_csv('exports/*.csv');
-- deux sources dans une requête
SELECT * FROM demo.ca_mensuel m
  JOIN objectifs.objectifs o USING (mois);
COPY (…) TO 'r.parquet' (FORMAT parquet);
```

Touches

F5 · Ctrl+R	exécuter la sélection, sinon le script
F6	profiler la requête sous le curseur
Ctrl+Espace	complétion ; « table. » : colonnes
Ctrl+B	commenter / décommenter
Ctrl+S · N · O	enregistrer · nouveau · ouvrir
Ctrl+Tab	script suivant

Connecteurs

DuckDB · mémoire	fichier .duckdb · base vide
Quack	hôte, port 9494, jeton (DuckDB 2.0)
SQLite	fichier
PostgreSQL · MySQL	hôte, port, base, utilisateur, mot de passe
SQL Server	serveur, base, utilisateur (mssql)
ODBC	DSN ; odbc_query, odbc_scan
CSV · Parquet · JSON · Excel	dossier + motif : une vue par fichier

Lire un plan

SCAN	lecture ; filtres poussés ?
HASH_JOIN	sortie ≫ entrées : jointure explosive
NESTED_LOOP	produit cartésien : condition manquante
GROUP_BY	regroupement ; nombre de groupes
ORDER · TOP_N	tri complet · n premiers
pastille · jauge	part du temps total

Recommandations

Critique	−15 : cartésien, scan de 10 M+
Important	−7 : scan 1 M+, jointure explosive, tris répétés, disque, requête répétée
Opportunité	−2 : clé primaire, colonnes inutiles, compression, agrégats
Mutualisation	empreintes : jointure commune, même grain
Correction · Ignorer	SQL dans une requête · oubli pour le projet

Pièges à éviter

- ▲ Sans sélection, F5 exécute tout le script : seul le dernier résultat s'affiche.
- ▲ Une table se préfixe de sa connexion : demo.ventes, pas ventes.
- ▲ L'affichage s'arrête à 1 000 lignes : Compter et Exporter voient tout.
- ▲ Une correction modifie la base : relisez-la, préférez la lecture seule pour analyser.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de la DuckDB Foundation



Aide-mémoire · Concevoir et automatiser

Modéliser, dessiner des pipelines, planifier, restituer, étendre.

Voir aussi : Chapitres 5 à 8

16 étapes de pipeline

ENCHAÎNEMENTS `SQL → pipeline` → `Valider` → `Aperçu` → `Exécuter` → `+ Travail (quotidien)` → `Démarrer` `F5` → `BI : Depuis le résultat` → `genre, mesures` → `Appliquer`

Schéma

Charger	modèle d'une connexion et d'un schéma
pastille → colonne	clé étrangère ; → titre : vers la PK
Suppr	table (après l'impact) ou clé
Impact	clés, vues, historique, scripts
Modèle → SQL	CREATE dans l'ordre des dépendances
Script → modèle	rejoue le DDL dans une base jetable

Étapes de pipeline

Sources	Table · Fichier · Requête SQL
Clauses	Filtre · Colonnes · Calcul · Agrégation · Tri · Limite · Dédoublonnage
Plusieurs entrées	Jointure · Union · SQL libre {entree}
Cibles	Table (remplacer, ajouter) · Fichier · Vue
Fusion	une chaîne = un SELECT ; étape partagée = WITH

Travaux planifiés

Exécute	pipeline · requête SQL · script .sql
Déclencheurs	manuel · toutes les N min · chaque jour à HH:MM · après
Dépend de	prérequis réussis, sinon « bloqué »
Reprises	n tentatives, attente en secondes ; ↻
Gantt	vert réussi · rouge échec · gris prévu

BI et assistant

Genres	colonnes, lignes, aires, secteurs, mosaïque, indicateur, tableau
Agrégats	somme, moyenne, nombre, min, max, distincts, aucun
Actions	question, SQL, plan, réécrire, erreur, modèle
Moteurs	règles · GGUF local · HTTP compatible OpenAI
Modèle (règles)	table : col type, … → autre_table

Extension minimale

```
fonction EXTENSION_MAISON()
retourner {nom = "Maison", ;
  regles = @MAISON_REGLES}

fonction MAISON_REGLES(ctx)
  locale l = {}
  pour chaque o dans ctx.objets
    si commencepar(o.nom, "tmp_")
      inserer(l, RECO("Opportunité", ;
        "Gouvernance", o.nom, "…", "", "…", nul))
    finsi
  suivant
retourner l
```

Pièges à éviter

- ▲ Le planificateur ne tourne que si DuckDB Studio est ouvert et démarré.
- ▲ Un filtre après un tri crée un nouveau bloc (WITH) : c'est voulu.
- ▲ Les extensions s'exécutent avec vos droits : confiance après lecture.
- ▲ L'assistant ne voit pas les données : précisez la question.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · DuckDB Studio, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr · DuckDB est une marque de la DuckDB Foundation